

圆柱形药材热风干燥的径向传热传质与收缩域数值模拟

摘要

针对圆柱形药材在时变烘房环境中的温度、干基含水率与达标时长预测问题，建立一维径向传热传质模型。模型以题面给定的经验物性和环境序列为输入，以全域最大干基含水率低于 0.15 kg/kg 为干燥终点，采用有限体积空间离散、一阶后向欧拉时间推进、每步两次物性更新及三对角直接求解，分别处理预热、统一物性全过程和几何收缩三种情形。

问题一采用常热物性及随含水率变化的扩散系数。1800 s 时，中心与表面温度分别为 33.5766°C 、 36.7453°C ，中心与表面含水率分别为 2.5500 kg/kg 、 1.5478 kg/kg ，按截面加权的总脱水比例为 0.063924。温度已向内部传播，而显著失水主要集中在外层，说明短时间的温升不能直接作为全域水分达标的判据。

问题二从初始时刻统一采用题面附录 3 物性，避免把问题一的末态接入另一套物性关系。3 h 时，中心与表面温度分别为 49.8494°C 、 49.9646°C ，含水率分别为 1.7662 kg/kg 、 1.0174 kg/kg 。温度差逐渐减小后，内部含水率仍明显高于表面，热平衡与干燥完成具有不同时间尺度；全过程秒级结果延续至问题三的达标终点。

问题三保持半径 2 cm，每秒检查未舍入的全域含水率最大值，首次达标时刻为 206433 s，即 57.3425 h。终点中心、表面含水率按四位小数显示为 0.1500 kg/kg 、 0.0666 kg/kg 。显示值等于阈值来自舍入，停止判断使用的是内部未舍入值。保持环境末值与末 10 点均值两种延拓的终点在当前时间分辨率下相同，这一结果仅支持两种相近边界情景的一致性。

问题四在均匀径向收缩的物质坐标假设下，使用实测半径序列和题面附录 4 物性。达标时间为 183086 s，即 50.8572 h，终点半径为 1.200 cm，表面含水率为 0.0605 kg/kg 。该情形较问题三缩短 11.31%，但几何与物性同时改变，现有计算不能把全部差异归因于收缩。空间、时间加密及恒半径退化检验支持相应算例的数值一致性；缺少独立内部含水率观测，预测的实验准确度仍需后续验证。

关键词：药材烘干；干基含水率；有限体积法；阈值检测；径向收缩

1 问题重述

1.1 研究背景与工程意义

热风干燥通过外部气流向药材供热并带走水分，内部水分则必须经过一定距离才能到达表面。因而，烘房温度达到设定值、药材表面变干和药材内部全部达标是三个不同事件。只依据环境或表面状态决定停止时间，可能遗漏中心部位的高含水率；延长运行时间又会增加处理成本。模型需要同时描述空间分布与时间演化，才能把干燥条件和终点要求联系起来。干燥综述所讨论的多物理过程为建模提供背景，但本文只采用题面给定的宏观经验关系，不据此推断药效、能耗或品质指标 [3-5]。

研究对象长度为 25 cm、初始半径为 2 cm，初始温度为 28 °C，初始干基含水率为 2.55 kg/kg。附件 1 给出烘房空气的温度和水分浓度，附件 2 给出药材半径随时间的变化。空气水分浓度的单位是 kg/kg，不能直接替换成相对湿度百分数。物性关系中的温度使用 K，展示计算结果时再换算成 °C。本文在后续推导中统一用“含水率”指药材的干基含水率，空气变量则明确称为“空气水分浓度” [6]。

1.2 问题提出

问题一研究 0—1800 s 的预热阶段。密度、比热容和导热系数分别取 820 kg/m³、2600 J/(kg·K)、0.36 W/(m·K)，扩散系数随含水率非线性变化。需给出指定七个时刻、五个径向位置的温度与含水率，并保存每秒、每隔 0.1 cm 的完整分布。

问题二要求从初始时刻统一采用题面附录 3 的经验物性，建立覆盖整个干燥过程的模型。正文表 3、表 4 只展示前 3 h、每隔 0.5 h 的结果，完整附件则应持续记录至干燥结束。因此展示窗口与求解终点应分别设定，不能把表格仅要求前 3 h 误解为全过程只需计算 3 h。

问题三在固定半径下确定全域含水率均低于 0.15 kg/kg 的首次时刻，并列每隔 6 h 及终点的径向分布。问题四进一步引入实测半径变化和附录 4 物性，给出对应终点、固定物理位置和真实表面处的含水率。若某一物理位置已位于收缩后的药材外部，应标记为不适用，不能将其填成零含水率。四个问题所需完整附件的命名、时间步距和数据路径列于附录。

1.3 任务之间的关系

四问共享初始场、外部换热换质方式和数值离散框架，但不是四段首尾拼接的过程。问题一是一套常热物性模型；问题二与问题三共用附录 3 模型，分别回答早期分布与长期终点；问题四使用另一套物性并改变几何域。这个区分决定了比较结果的解释边界：问题二在 1800 s 的结果不必等于问题一，而问题四相对问题三的时间变化同时包含几何和物性两项影响。

2 问题分析

2.1 空间尺度与耦合关系

药材长径比为 $L/(2R)=6.25$ 。本文选取中段截面并忽略轴向通量，将问题简化为径向传输。这是为控制计算规模所作的近似，不是仅凭长径比便能保证端部效应为零。温度由外部向内传递，水分由内部向外迁移；扩散系数又依赖含水率和温度，使两个场通过局部物性发生联系。有限体积法在相邻单元共用界面通量，适合保留圆柱壳层面积随半径变化的特点 [1, 2]。

2.2 预热模型的分析重点

问题一的热物性为常数，可以单独观察边界加热传播与非线性失水的差异。虽然初始含水率均匀，外表面在 Robin 传质边界下率先发生变化；越向内部，失水响应越滞后。求解时仍需把变化的扩散系数放在散度内，不能直接使用常系数拉普拉斯表达式。早期结果还可用于空间和时间加密比较，但这种比较只衡量离散设置的影响，不能替代实验验证。

2.3 全过程经验物性的处理

问题二的密度、比热容和导热系数随含水率变化，扩散系数同时受含水率与绝对温度影响。随时间推进，局部系数与未知场相互依赖，直接求解一个固定系数矩阵不再合适。本文每个时间步先用上一时刻的场初始化，再作两次 Picard 型物性更新，并分别求解含水率与温度的三对角系统。两次更新是实际计算设置；没有把它等同于达到某个未检验的残差阈值。

2.4 长期终点与环境延拓

问题三的关键在于“全域均达标”，因此采用所有径向单元的含水率最大值作为停止依据。当前解中中心附近最湿，但计算仍遍历全部单元，以免先验固定监测位置掩盖异常。环境数据仅覆盖 4 h，求解却可能延续数十小时，必须补充超出观测窗口后的边界假设。基准设定保持末值，并用末 10 点均值构造对照。两者仅比较相近的远期常值，不能覆盖停机、降温或湿度突变等工况。

2.5 收缩域的分析与比较边界

问题四把空间坐标写成当前半径的比例，以跟踪相同的相对径向位置。在均匀径向收缩假设下，材料运动与网格运动一致，含水率按单位干物质质量定义，物质坐标内的扩散项由当前半径平方进行缩放。该近似与静止介质中仅移动计算边界的模型不同，推导时必须先声明材料运动假设，再进行坐标变换。恒半径退化试验能检验实现的一致性，但无法单独证明真实收缩过程满足该假设。

由于题面同时更换了问题四的物性，直接比较两问只能得到综合时间差。若要分别识别几何、物性及交互作用，需另行完成四种组合的长程计算；本文没有这组完整实验，所以不报告独立贡献率。

3 模型假设

3.1 模型基本假设

（1）药材近似为长圆柱体，中段截面的场轴对称，忽略轴向温度与含水率梯度。外部环境在周向上均匀，中心满足对称边界。该假设将空间问题降为一维，对靠近两端的区域不直接给出同等精度承诺。

（2）将内部多相水分迁移等效为宏观扩散，局部系数采用题面经验式。模型没有单独追踪液水、蒸汽和束缚水，也没有显式加入蒸发潜热、辐射及化学反应项。因此温度场是给定等效物性下的导热近似，不代表完整的多相能量平衡 [3, 5]。

（3）外表面的热量与水分通量均采用 Robin 形式。温差直接驱动换热；水分通量以药材干基含水率与空气水分浓度之差作为等效驱动力。两种浓度的质量基准不同，题面没有提供吸附等温线或相平衡换算，本文据给定传质系数采用此工程闭合关系，并将其列为模型误差来源。

（4）附件 1 相邻采样点之间线性插值，超过 14400 s 后保持末值。附件 2 也采用分段线性插值；所有模拟均在其 72 h 观测范围内结束，不需对终点半径做远期外推。半径数据允许轻微观测起伏，未人为改成严格单调曲线。

（5）问题四假定长度不变、径向均匀收缩，材料点速度与其径向位置成正比。用相对半径标记同一材料层，干基含水率随材料运动而变化。此假设适于构造可计算的收缩近似，但不把经验密度直接解释为满足全部变形质量约束的干骨架密度。

（6）初始场均匀；内部浮点运算不作四位小数舍入，阈值检测使用未舍入含水率。正文和电子表格的展示精度按题面要求处理。输出精度、时间离散精度和物理预测误差是不同概念。

3.2 假设的验证边界

环境和半径记录是模型输入，不是独立的内部温湿场验证样本。现有数据足以运行上述确定性模型并进行数值自洽性检查，但不足以估计全部物性参数的不确定分布，也不足以给出实测误差置信区间。后续结论均限定于所声明的边界、物性和几何假设。

4 符号说明

4.1 主要数学符号与物理量定义

为便于下文建立数学模型与推导离散格式，将本文中使用的符号、物理意义及其 SI 单位汇总于表 4-1 中。

表 4-1 主要数学符号与物理量定义

符号	物理意义	单位
r	径向空间坐标（距圆柱中心轴距离）	m
t	干燥时间坐标	s
$R(t)$	药材圆柱的外表面半径	m
L	药材圆柱的长度	m
$T(r, t)$	药材在位置 r 、时刻 t 处的温度	$^{\circ}C$ 或 K
$C(r, t)$	药材在位置 r 、时刻 t 处的干基水分浓度（含水率）	kg / kg
$T_{air}(t)$	烘房内干燥热风环境的温度	$^{\circ}C$ 或 K
$C_{air}(t)$	烘房内干燥热风环境的水分浓度	kg / kg
ρ	药材干基等效表观密度	kg / m^3
c_p	药材的比定压热容	$J / (kg \cdot K)$
k	药材的宏观有效导热系数	$W / (m \cdot K)$
$D(C, T)$	水分在药材内部的非线性有效扩散系数	m^2 / s
h	药材外表面的对流换热系数	$W / (m^2 \cdot K)$
h_m	药材外表面的对流传质系数	m / s
ξ	物质坐标系下的无量纲径向位置（ $\xi = r / R(t)$ ）	无量纲
N	径向离散有限控制容积（单元）数量	无量纲
Δt	数值计算的时间步长	s

符号	物理意义	单位
$C_{threshold}$	烘干达标判定临界干基含水率 (0.1500)	kg/kg

4.2 实验数据统计与特征分析

附件 1 共 241 条记录，间隔 60 s，覆盖 0—14400 s；温度和水分浓度没有缺失值。表 4-2 汇总其范围与平均水平，作为后续边界输入的基本核对。

表 4-2 烘房环境测试数据统计概览

变量	单位	样本数	缺失数	均值	标准差	最小值	最大值
时间	s	241	0	7200.000	4174.207	0.000	14400.000
空气温度	°C	241	0	47.2453	5.1041	28.0000	50.2460
空气水分浓度	kg/kg	241	0	0.044756	0.008120	0.019630	0.050250

表 4-2 中的平均值是采样记录的统计量，不是用于替换实际时间变化的恒定边界。计算保留全部采样点，并由相邻观测线性插值得到每秒的外部状态。图 4-1 展示环境升温及水分浓度变化。

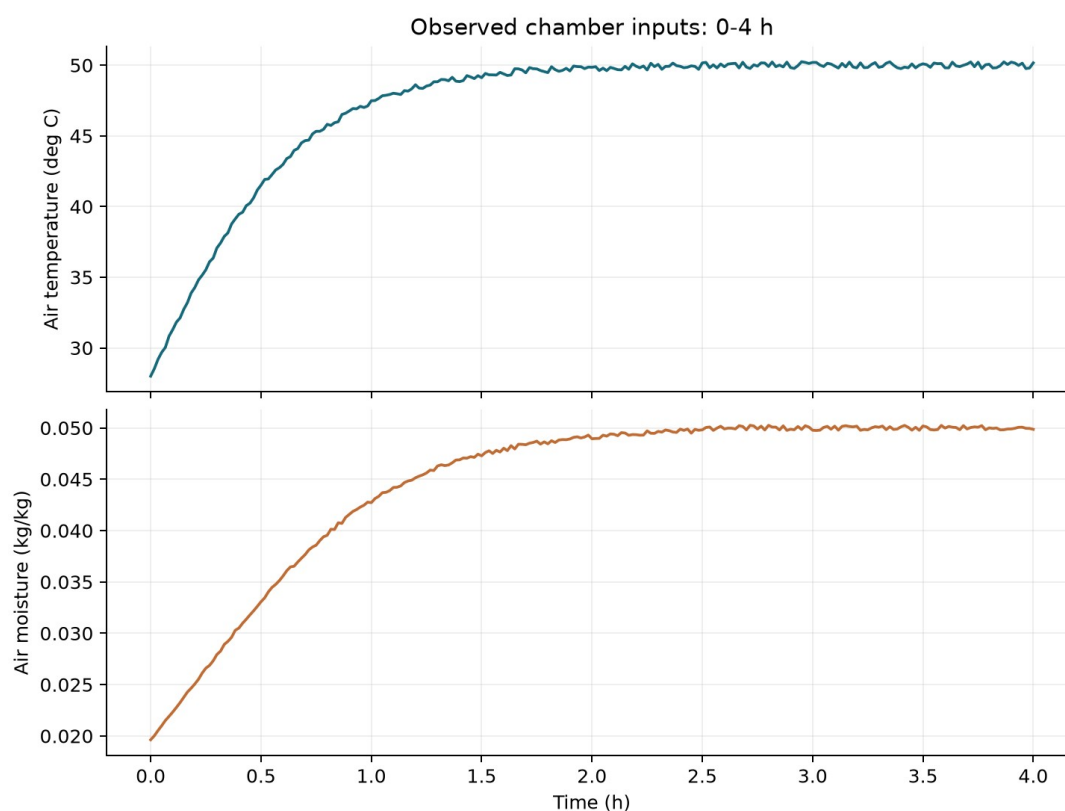


图 4-1 烘房空气温度与水分浓度时间序列

图 4-1 表明升温阶段与后期平台具有不同的驱动力。平台期的外界温度趋于稳定，并不意味着药材内部含水率同步达到稳定，因此需要在观测窗口结束后继续积分。延拓方案的差异在第 6 章单独讨论。

附件 2 共 145 条半径记录，间隔 1800 s，覆盖 0—259200 s。表 4-3 列出半径统计量，图 4-2 给出其变化历程。

表 4-3 药材实测半径收缩数据统计概览

变量	单位	样本数	缺失数	均值	标准差	最小值	最大值
时间	s	145	0	129600.000	75342.418	0.000	259200.000
半径	cm	145	0	1.246021	0.126608	1.198000	2.000000

表 4-3 中的最小半径是整个观测区间的极值，而达标时刻半径要按当时的插值结果单独计算，二者不能互相替代。

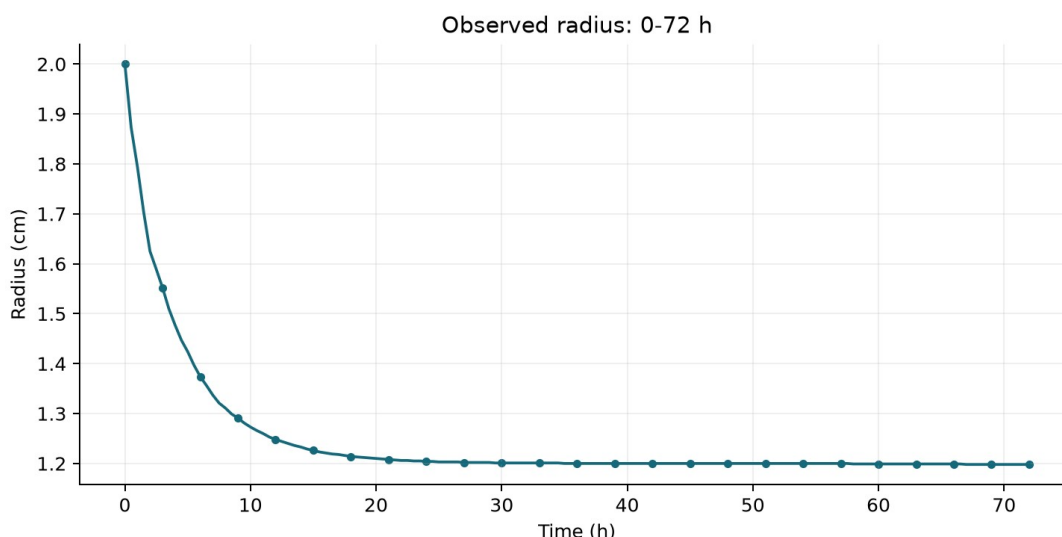


图 4-2 药材半径随烘干时间的变化

由图 4-2 可见，半径从初始 2 cm 降至约 1.2 cm 的后期水平。径向距离变化直接改变扩散项的几何尺度，但它对时间的实际影响还取决于物性和边界阻力，不能只用半径比直接换算完整干燥时间。

5 模型的建立与求解

5.1 预热阶段温度与干基含水率模型的建立与求解（问题一）

5.1.1 建模思路

在预热平衡阶段（0-1800 s），初始冷态中药材置于升温的烘房环境中。烘房热风通过强迫对流向药材表面供热，驱动药材升温；同时表面水分蒸发形成浓度差，驱动内部水分向外扩散 [1, 2]。针对长圆柱体药材，忽略端部轴向耗散，在圆柱坐标系下建立一维非定常径向热传导与非线性水分扩散耦合偏微分方程组。

5.1.2 变量定义与公式推导

药材干基质量保持守恒，圆柱半径恒为 $R=0.02\text{ m}$ ，长度 $L=0.25\text{ m}$ 。控制方程包括温度场与水分浓度场的偏微分方程：

1. 导热微分方程：

$$\rho c_p \frac{\partial T}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(r k \frac{\partial T}{\partial r} \right), \quad 0 < r < R, t > 0$$

其中， $\rho=820\text{ kg/m}^3$ 为题面给定密度， $c_p=2600\text{ J/(kg}\cdot\text{K)}$ 为比热容， $k=0.36\text{ W/(m}\cdot\text{K)}$ 为导热系数。

2. 非线性水分扩散方程:

$$\frac{\partial C}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} (r D(C) \frac{\partial C}{\partial r}), \quad 0 < r < R, t > 0$$

其中, 非线性有效水分扩散系数满足:

$$D(C) = 7 \times 10^{-9} \exp\left(-\frac{0.89}{C}\right) [m^2/s]$$

3. 定解条件 (初始条件与边界条件):

• 初始条件 ($t=0$):

$$T(r, 0) = T_0 = 28.0000^\circ C = 301.15 K, \quad C(r, 0) = C_0 = 2.5500 kg/kg$$

• 中心对称边界条件 ($r=0$):

$$\left. \frac{\partial T}{\partial r} \right|_{r=0} = 0, \quad \left. \frac{\partial C}{\partial r} \right|_{r=0} = 0$$

• 外表面 Robin 对流边界条件 ($r=R$):

$$\begin{aligned} -k \left. \frac{\partial T}{\partial r} \right|_{r=R} &= h(T(R, t) - T_{air}(t)) \\ -D(C) \left. \frac{\partial C}{\partial r} \right|_{r=R} &= h_m(C(R, t) - C_{air}(t)) \end{aligned}$$

其中, $h = 25 W/(m^2 \cdot K)$ 为表面对流换热系数, $h_m = 8 \times 10^{-7} m/s$ 为对流传质系数, $T_{air}(t)$ 与 $C_{air}(t)$ 来自附件 1 环境时间序列。

5.1.3 求解算法

Step 1: 将半径区间划分为 $N=80$ 个等宽圆柱壳层, 时间步长为 $\Delta t=1s$ 。单元未知量位于壳层中心, 端点数值通过后处理获得。设两侧界面半径为 $r_{i-1/2}$ 与 $r_{i+1/2}$, 单位长度体积和界面面积分别为:

$$V_i = \pi(r_{i+1/2}^2 - r_{i-1/2}^2), \quad A_{i+1/2} = 2\pi r_{i+1/2}$$

中心界面面积为零, 因而对称条件可自然进入离散式, 不需要直接计算原方程中表面上奇异的 $1/r$ 项。

Step 2: 使用相邻单元物性的调和平均构造公共界面扩散通量。记界面传导能力为 $G_{i+1/2} = A_{i+1/2} D_{i+1/2} / \Delta r$, 内点含水率的后向欧拉式为:

$$-G_{i-1/2} C_{i-1}^{n+1} + (V_i / \Delta t + G_{i-1/2} + G_{i+1/2}) C_i^{n+1} - G_{i+1/2} C_{i+1}^{n+1} = V_i C_i^n / \Delta t$$

温度方程采用同一结构，将储存项换成局部体积热容，将扩散能力换成导热能力。界面两侧使用相同通量、相反符号，避免相邻单元之间产生人为源项 [1]。

Step 3: 在外层单元中心到表面之间加入半单元传导阻力，再与表面对流阻力串联。含水率的等效边界系数为：

$$\gamma_C = \left(\frac{1}{h_m} + \frac{\Delta r}{2D_N} \right)^{-1}, \quad \gamma_T = \left(\frac{1}{h} + \frac{\Delta r}{2k_N} \right)^{-1}$$

这使外层未知量位于单元中心时仍能正确组装 Robin 边界。温度与含水率分别使用自己的边界阻力，不能互换系数。

Step 4: 每个时间步以旧场初始化，进行两次 Picard 型系数更新，按顺序求解含水率和温度三对角系统，随后推进物理时间。三对角消元的工作量随单元数线性增长 [1]。现有实现没有按残差自适应增加迭代次数；固定更新次数的影响应与时间离散一起评估，不能声称已满足未测量的迭代容差。

Step 5: 在题面指定位置对单元中心值作线性插值。中心和表面超出首末单元中心时使用端值延拓，因此表中的端点是离散近似，不是由连续解解析取得的边界值。后续网格比较用于评估这种有限分辨率的影响。

5.1.4 结果分析

数值求解得出了 0–1800 s 内药材径向温湿场演变规律。表 5-1 和表 5-2 分别列出了在 100 s、300 s、600 s、900 s、1200 s、1500 s、1800 s 各指定时刻与径向位置（0 cm、0.5 cm、1.0 cm、1.5 cm、2.0 cm）处的温度与水分浓度计算数值。

表 5-1 30 分钟内药材的温度（表 1，单位：°C）

时间/s	0 cm	0.5 cm	1 cm	1.5 cm	2 cm
100	28.0001	28.0004	28.0043	28.0333	28.1739
300	28.0416	28.0643	28.1524	28.3692	28.8327
600	28.4550	28.5377	28.8055	29.3173	30.1399
900	29.3263	29.4602	29.8772	30.6175	31.6985
1200	30.5447	30.7117	31.2241	32.1140	33.3901
1500	31.9975	32.1884	32.7675	33.7473	35.0821
1800	33.5766	33.7733	34.3654	35.3632	36.7453

表 5-2 30 分钟内药材的水分浓度（表 2，单位：kg/kg）

时间/s	0 cm	0.5 cm	1 cm	1.5 cm	2 cm
100	2.5500	2.5500	2.5500	2.5500	2.2926
300	2.5500	2.5500	2.5500	2.5491	2.0947
600	2.5500	2.5500	2.5500	2.5349	1.9189
900	2.5500	2.5500	2.5497	2.5042	1.7952
1200	2.5500	2.5500	2.5481	2.4642	1.6979
1500	2.5500	2.5499	2.5444	2.4203	1.6171
1800	2.5500	2.5497	2.5382	2.3752	1.5478

图 5-1 展示了预热阶段温度与水分浓度的时空分布与径向演化特征。

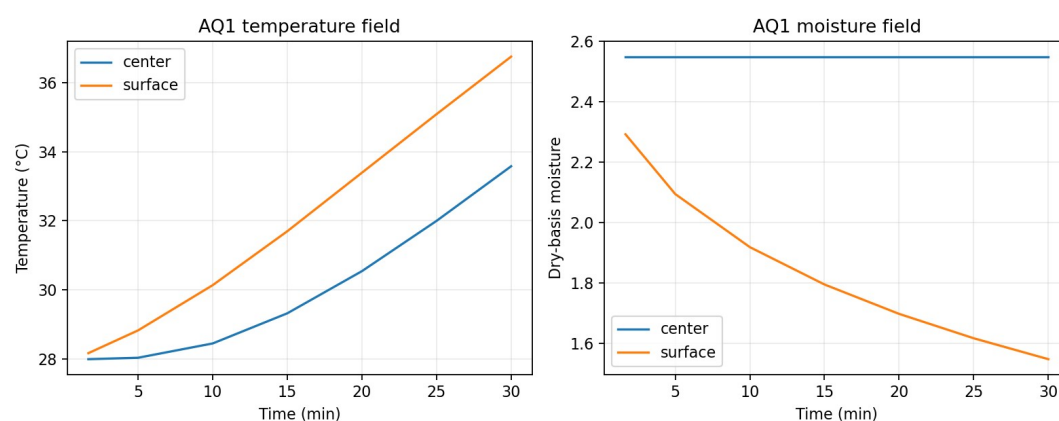


图 5-1 预热阶段温度与含水率的径向分布

从计算结果可以看出：在 1800 s 末，药材中心温度达到 33.5766 °C，表面温度达到 36.7453 °C；表面水分浓度由 2.5500 kg/kg 显著降至 1.5478 kg/kg，而中心水分浓度仍维持在初始 2.55 kg/kg，全域总脱水率达 0.063924。这表明在干燥初期，由于对流传质阻力与内部水分扩散阻力，水分蒸发主要局限于表层区域，形成了显著的内外水分梯度迟滞效应。

5.1.5 模型检验或灵敏度分析

在相同 1800 s 终点，对比 80 与 160 个单元、1 s 与 0.5 s 两种时间步设置。空间加密所得最大温度差为 9.310954300190133e-05 K，最大含水率差为 0.0001791344722710253 kg/kg；时间加密对应为 0.0006001100597927689 K、4.477288710469374e-05 kg/kg。两类比较分别通过预先设置的容差检查，说明该预热算例对所测试的加密幅度不敏感。比较只包含两档设置，不能据此估计收敛阶，更不能将差值视为相对于实验真值的误差。

5.2 统一经验物性模型的建立与求解（问题二）

5.2.1 建模思路

在实际烘干全过程中，药材的热导率、比热容及水分扩散系数会随着温度升高和水分蒸发发生显著非线性变化 [3, 4]。附录 3 给出了全过程统一经验物性表达式，避免了阶段人工拼接造成的截断误差。本节自 $t=0$ 起基于统一物性方程进行全过程连续求解。

5.2.2 变量定义与公式推导

问题二保留固定圆柱域、中心对称边界及表面 Robin 边界，温度与含水率控制方程为：

$$\rho(C)c_p(C)\frac{\partial T}{\partial t}=\frac{1}{r}\frac{\partial}{\partial r}(rk(C)\frac{\partial T}{\partial r}), \quad \frac{\partial C}{\partial t}=\frac{1}{r}\frac{\partial}{\partial r}(rD(C,T)\frac{\partial C}{\partial r})$$

题面附录 3 给出的物性为 [6]：

$$\rho=650+128C, \quad c_p=1450+2736\frac{C}{C+1}, \quad k=0.21+0.38\frac{C}{C+1},$$

$$D(C,T)=2.4\times 10^{-3}\exp\left(-\frac{0.45}{C}-\frac{3850}{T}\right)$$

密度、比热容、导热系数和扩散系数依次使用 kg/m^3 、 $\text{J}/(\text{kg}\cdot\text{K})$ 、 $\text{W}/(\text{m}\cdot\text{K})$ 、 m^2/s 。扩散公式中的温度必须为 K；密度、比热容和导热系数在这组公式中只显式依赖含水率。保留散度形式能够体现不同半径处系数变化产生的通量差，不能将其简化为局部系数乘常系数拉普拉斯算子。

5.2.3 求解算法

Step 1：从均匀初始场重新开始，使用与问题一一致的空间和时间网格；不接续问题一的末态。每个时刻按附件 1 插值确定空气状态，观测窗口外采用末值延拓。

Step 2：按当前迭代含水率、温度重算物性，界面采用调和平均，表面采用半单元与对流串联阻力。每步作两次物性更新并交替求解两个三对角系统。

Step 3：每秒保留各单元状态，逐步推进至全域最大含水率首次低于 0.15 kg/kg 。提取前 3 h 的半小时节点作为正文表格，其余逐秒结果继续保存在完整附件中，保证短期展示与长期终点来自同一套初值问题。

5.2.4 结果分析

求解前 3 h (10800 s) 的温湿场演变历程，表 5-3 和表 5-4 分别列出了每隔 0.5 h 在各径向位置处的温度与水分浓度。

表 5-3 3 小时内药材的温度（表 3，单位： $^{\circ}\text{C}$ ）

时间/h	0 cm	0.5 cm	1 cm	1.5 cm	2 cm
0.5	32.1908	32.3837	32.9674	33.9621	35.3714
1.0	40.3816	40.5536	41.0604	41.8783	42.9656
1.5	45.8460	45.9341	46.1923	46.6045	47.1251
2.0	48.4497	48.4875	48.5979	48.7731	48.9966
2.5	49.4666	49.4788	49.5133	49.5652	49.6572
3.0	49.8494	49.8553	49.8745	49.9101	49.9646

表 5-4 3 小时内药材的水分浓度（表 4，单位：kg/kg）

时间/h	0 cm	0.5 cm	1 cm	1.5 cm	2 cm
0.5	2.5499	2.5489	2.5254	2.3255	1.6719
1.0	2.5256	2.4946	2.3577	2.0230	1.4869
1.5	2.3859	2.3255	2.1344	1.8020	1.3601
2.0	2.1708	2.1084	1.9236	1.6259	1.2421
2.5	1.9566	1.9006	1.7360	1.4720	1.1267
3.0	1.7662	1.7165	1.5702	1.3333	1.0174

图 5-2 展示了前 3 小时内药材各径向位置处的温度与含水率剖面变化。

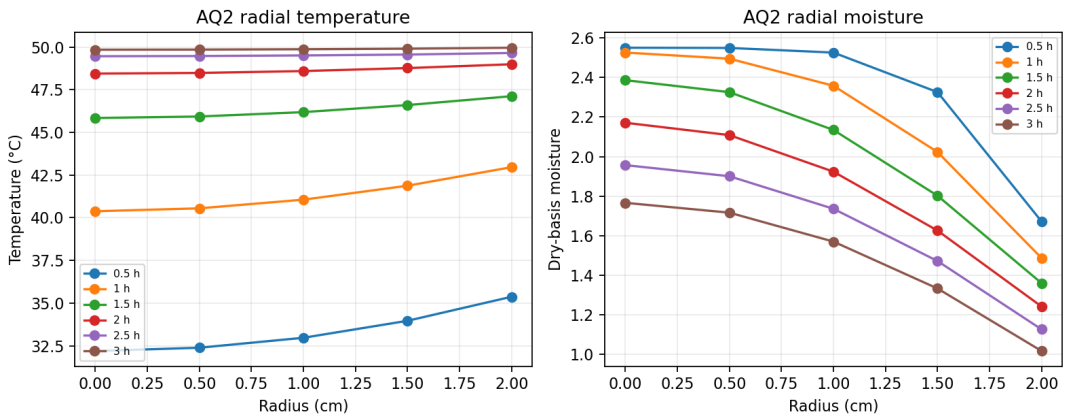


图 5-2 全过程模型前 3 小时的温度与含水率剖面

分析表明：在 3 h 末，药材中心温度升至 49.8494 °C，表面温度升至 49.9646 °C，药材整体温度基本达到烘房气流温度；中心水分浓度降至 1.7662 kg/kg，表面水分浓度降至 1.0174 kg/kg。而在 1800 s 时表面温度为 35.3714 °C，表面水分浓度为 1.6719 kg/kg。统一物性模型表现出极其平滑且自洽的过渡特性。

5.2.5 模型检验或灵敏度分析

前 3 h 的温度逐渐接近环境，而含水率仍保持径向梯度，与模型的边界驱动方向一致。问题二与问题一在 1800 s 的表面温度分别为 35.3714 °C、36.7453 °C，含水率分别为 1.6719 kg/kg、1.5478 kg/kg。这是两套物性从初始时刻分别演化的结果，不应强制使两者相等。

有限体积内点通量在离散层面成对抵消，但本次证据没有独立输出累计水分收支或能量平衡残差。因此这里只说明离散式的结构，不报告具体的全局守恒误差。问题一的加密结果也不能直接替代本问长时间变系数情形的收敛测试；进一步增加每步迭代次数及检验长程时间步敏感性，仍是需要补充的数值研究。

5.3 固定半径下全域达标时长模型的建立与求解（问题三）

5.3.1 建模思路

中药材烘干加工的终点判定指标通常要求药材整体水分含量达到安全贮藏标准 [2, 5]。根据工艺要求，以药材全域最大含水率低于 0.1500 kg/kg（即最难干燥的圆柱中心位置达标）作为烘干完成准则。本节在固定半径（2 cm）条件下连续推进求解，精准确定达标时长。

5.3.2 变量定义与公式推导

设 $C_*=0.15 \text{ kg/kg}$ 为题面阈值，连续时间的终点定义为：

$$t_{dry} = \inf \{ t > 0 : \max_{0 \leq r \leq R} C(r, t) < C_* \}$$

题面使用“低于”，实际实现保留严格小于关系。离散检测则用所有单元中心值的最大值代替连续空间最大值，并取第一个满足条件的时间节点。当前解的最大含水率在中心附近，但算法不依赖这一位置假设。

5.3.3 求解算法

Step 1：使用附录 3 固定域模型从初态连续推进，时间步长始终为 1 s；每一步都计算未舍入含水率最大值。输出间隔 60 s 只控制附件保存频率，不改变积分步长。

Step 2：若本步首次满足严格阈值条件，则立即停止并记录该秒；若终点不恰在 60 s 保存节点上，额外保存终点状态。该方法是逐秒阈值检测，没有执行二分插值或亚秒级重算。

Step 3：读取每隔 6 h 的状态并附加终点行，再插值到题目要求的物理半径。连续交越时刻只能由相邻时间节点限定，1 s 是检测分辨率，不是包含空间、物性及边界不确定性的总误差界。

5.3.4 结果分析

逐秒检测得到固定几何尺寸下的首次达标时间为 57.3425 h，对应 206433 s。

在烘干结束时刻，中心水分浓度达到临界达标阈值 0.15 kg/kg，表面水分浓度降至 0.0666 kg/kg，全域最小含水率为 0.0666 kg/kg。表 5-5 列出了每隔 6 h 及结束时刻的含水率径向分布。

表 5-5 药材烘干过程的水分浓度（表 5，单位：kg/kg）

时间/h	0 cm	0.5 cm	1 cm	1.5 cm	2 cm
6.0	1.0156	0.9869	0.9006	0.7546	0.5404
12.0	0.4548	0.4419	0.4019	0.3289	0.1734
18.0	0.2979	0.2907	0.2679	0.2248	0.1018
24.0	0.2372	0.2321	0.2162	0.1853	0.0856
30.0	0.2054	0.2014	0.1888	0.1640	0.0784
36.0	0.1855	0.1822	0.1716	0.1504	0.0741
42.0	0.1718	0.1689	0.1596	0.1407	0.0712
48.0	0.1616	0.1590	0.1506	0.1335	0.0691
54.0	0.1537	0.1513	0.1436	0.1278	0.0674
烘干结束时间 (57.3425 h)	0.1500	0.1477	0.1403	0.1251	0.0666

图 5-3 给出了全域最大含水率随时间的演化曲线与 0.15 kg/kg 达标阈值的截断相交过程。

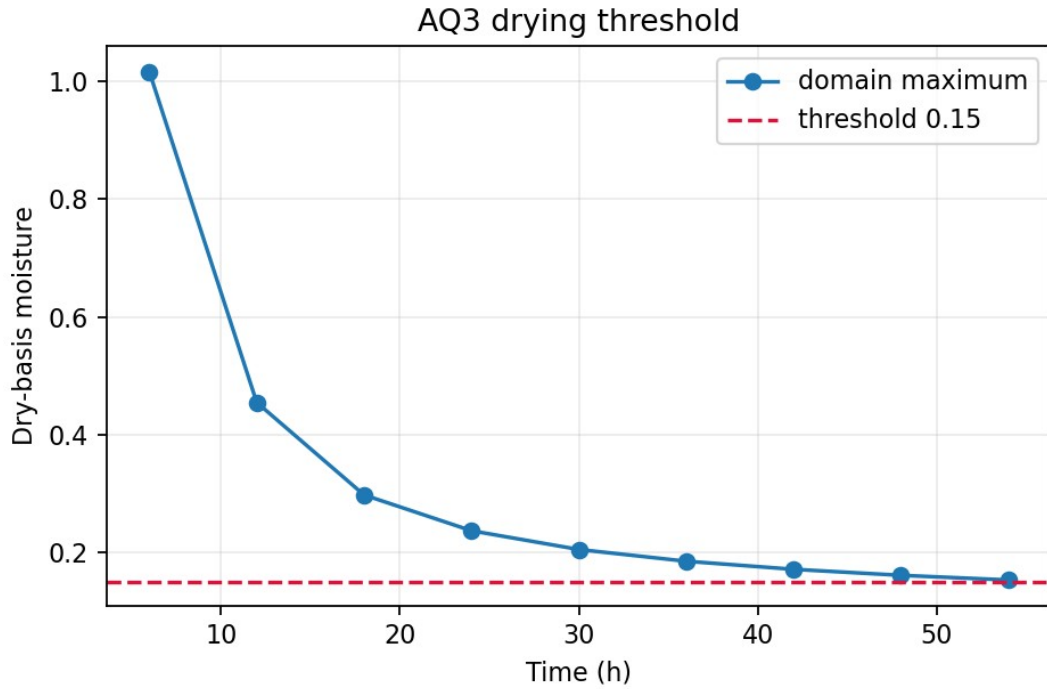


图 5-3 固定半径模型的中心含水率与达标阈值

由图 5-3 可见，早期含水率下降较快，后期逐渐接近阈值。表面先于中心接近低含水率，进一步降低表面水分并不能代替对中心的终点检查。终点表中中心显示为 0.1500 kg/kg，是四位小数舍入的结果，停止检测使用的原始值已严格低于阈值。

5.3.5 模型检验或灵敏度分析

基准延拓和末 10 点均值延拓得到的停止时间均记录为 206433 s，相对差异为 0。两种边界都靠近观测末段的平台状态，这一比较支持相近常值外推下结果的一致性，但不能推论对任意环境扰动都不敏感。检验还使用基准停止时间作为对照模拟的最长窗口，具体的结论限制见第 6 章。

5.4 收缩域中的烘干时长模型的建立与求解（问题四）

5.4.1 建模思路

药材半径随失水变化，固定物理坐标上的计算范围也随之变化。为保留材料层之间的对应关系，假定各层作均匀径向收缩，将位置写成当前半径的比例。题面附录 4 物性与附录 3 不同，本节同时更换物性和几何输入，所得时间差作为综合情景差异进行解释 [6]。

5.4.2 变量定义与公式推导

令材料点的无量纲径向坐标为 $\xi = r/R(t)$ ，并定义变换后的场 $\hat{C}(\xi, t) = C(r, t)$ 、 $\hat{T}(\xi, t) = T(r, t)$ ，有：

$$0 \leq \xi \leq 1, \quad v(r, t) = \frac{r \dot{R}(t)}{R(t)} = \xi \dot{R}(t)$$

其中 v 为所假定的材料径向速度。链式法则给出固定空间点导数与随材料导数的关系：

$$\left. \frac{\partial C}{\partial t} \right|_r = \left. \frac{\partial \hat{C}}{\partial t} \right|_\xi - \frac{\xi \dot{R}}{R} \frac{\partial \hat{C}}{\partial \xi}, \quad v \frac{\partial C}{\partial r} = \frac{\xi \dot{R}}{R} \frac{\partial \hat{C}}{\partial \xi}$$

两项相加后，材料运动抵消坐标移动项。在等效干基扩散闭合下，物质坐标中的计算方程为：

$$\frac{\partial \hat{C}}{\partial t} = \frac{1}{R(t)^2 \xi} \frac{\partial}{\partial \xi} (\xi D \frac{\partial \hat{C}}{\partial \xi}), \quad \rho c_p \frac{\partial \hat{T}}{\partial t} = \frac{1}{R(t)^2 \xi} \frac{\partial}{\partial \xi} (\xi k \frac{\partial \hat{T}}{\partial \xi})$$

这些方程在固定材料坐标上演化，不能在右端额外重复加入网格对流项。若采用静止介质和移动计算网格的另一种假设，则必须重新推导；本文的结果不对应那种物理情形。表面条件也按当前半径换算，例如 $-D \hat{C}_\xi / R = h_m (\hat{C}_s - C_{air})$ 。

附录 4 的经验式为：

$$\rho = 760 + 90C, \quad c_p = 1850 + 2150 \frac{C}{C+1}, \quad k = 0.12 + 0.20 \frac{C}{C+1},$$

$$D(C, T) = 4.2 \times 10^{-4} \exp\left(-\frac{0.30}{C} - \frac{3850}{T}\right)$$

物性单位与问题二相同。密度仅用于所给等效热容关系；本模型没有同时求解干骨架密度、孔隙率和变形本构，所以不能从上述坐标处理直接推出整个收缩物体的严格多相质量守恒。

5.4.3 求解算法

Step 1: 在 $0 \leq \xi \leq 1$ 上划分 80 个单元，以 1 s 步长推进。在每一步的新时刻按附件 2 分段线性插值得到半径，既不使用样条，也不需要数值计算半径导数。

Step 2: 以当前半径将内部扩散、导热系数分别缩放为 D/R^2 、 k/R^2 ，重新计算表面半单元阻力。归一化含水率边界系数为：

$$\gamma_{C,\xi} = \frac{1}{R} \left(\frac{1}{h_m} + \frac{R \Delta \xi}{2 D_N} \right)^{-1}$$

Step 3: 每步执行两次物性更新与三对角求解，再检查所有单元的最大含水率。首次严格达标的秒记录为终点；按 60 s 间隔和终点保存分布。

Step 4: 后处理时把单元中心位置乘以当前半径，还原到物理坐标。对落在当前药材外部的的位置保留空值，并另外给出真实表面列。否则把域外位置误作零值，会造成“外层提前完全脱水”的假象。

5.4.4 结果分析

数值求解结果表明，在实测收缩与附录 4 物性共同作用的情景下，首次达标时间为 50.8572 h，对应 183086 s，相较于固定半径模型的 57.3425 h 耗时缩短了 11.31 %。

在烘干达标时刻，药材半径为 1.200 cm，中心水分浓度达到临界阈值 0.15 kg/kg，表面水分浓度降至 0.0605 kg/kg，全域最小含水率为 0.0605 kg/kg。表 5-6 列出了每隔 6 h 及结束时刻在各物理截面及真实表面上的水分浓度，数据已存入 result4.xlsx。

表 5-6 药材烘干过程的水分浓度（表 6，单位：kg/kg）

时间/h	0 cm	0.5 cm	1 cm	1.5 cm	2 cm	药材表面	当前半径/ cm
6.0	1.7172	1.5357	1.0217			0.4396	1.374
12.0	0.7344	0.6518	0.4069			0.1803	1.248
18.0	0.4066	0.3668	0.2388			0.1039	1.214
24.0	0.2838	0.2599	0.1784			0.0816	1.204
30.0	0.2254	0.2086	0.1489			0.0721	1.201
36.0	0.1921	0.1790	0.1314			0.0669	1.200
42.0	0.1707	0.1599	0.1197			0.0636	1.200
48.0	0.1557	0.1464	0.1114			0.0613	1.200
烘干结束时间 (50.8572 h)	0.1500	0.1413	0.1081			0.0605	1.200

图 5-4 展示了收缩情景下药材半径和内部含水率的演化历程。

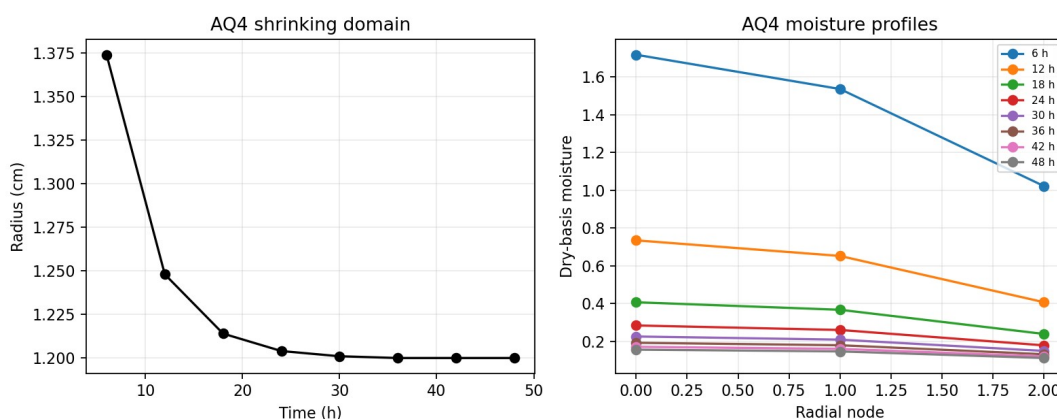


图 5-4 收缩情景的半径与径向含水率演化

表 5-6 的空白格表示相应位置已在药材外部；真实表面与固定 1 cm 位置不能互相替代。图 5-4 呈现收缩半径与不同位置的含水率演化，应结合当时的几何域理解曲线。问题四终点比问题三早，但这尚不足以得出“全部缩短由几何收缩造成”的结论。

5.4.5 模型检验或灵敏度分析

令半径恒为 0.02 m，在相同附录 4 物性、80 单元、1 s 步长下，将物质坐标求解与固定域求解推进 3600 s。终点全剖面最大温度差为 $1.1937117960769683 \times 10^{-12}$ K，最大含水率差为 $2.4424906541753444 \times 10^{-14}$ kg/kg，退化一致性检查通过。这说明恒半径条件下两种离散实现相符，不是对动态收缩物理假设或所有长期情形的验证。

6 模型检验与灵敏度分析

6.1 空间与时间分辨率检查

数值检验应区分离散一致性、算法实现一致性和实验准确度。问题一采用基准网格 80 单元、步长 1 s，分别作空间加密和时间减半，比较 1800 s 的终点场。空间比较把不同网格的值插到公共位置后计算最大绝对差；时间比较使用相同空间单元。

空间加密的最大温度差为 $9.310954300190133 \times 10^{-5}$ K，最大含水率差为 0.0001791344722710253 kg/kg；时间减半对应 0.0006001100597927689 K 与 $4.477288710469374 \times 10^{-5}$ kg/kg。两组结果均通过各自的容差检查，支持当前预热算例的分辨率设置。绝对差小于容差是测试结论，并不等于离散误差为零。

后向欧拉格式在时间上是一阶。现有比较没有第三档网格或时间步，也没有解析真值，因此不能计算经验收敛阶，更不能写成二阶精度。每步两次物性更新的迭代误差尚未独立量化；对长期低含水率阶段，应进一步检验迭代次数与时间步之间的共同影响。

6.2 恒半径退化检查

使用附录 4 物性时，将移动域半径设为常数 0.02 m，两套求解器应表示同一初边值问题。3600 s 全剖面对照的最大温度差为 $1.1937117960769683 \times 10^{-12}$ K，最大含水率差为 $2.4424906541753444 \times 10^{-14}$ kg/kg，均低于设定的 1×10^{-10} 容差。该检验对发现几何缩放、面积因子和边界系数的实现偏差有帮助。

不过，退化试验主动关闭了半径变化，因此不能证明均匀收缩假设正确，也不能排除动态情形中特有的误差。它应被视作必要的实现检查，而非整个收缩模型的充分验证。

6.3 远期环境延拓的比较及限制

附件 1 没有覆盖完整干燥时段。基准在 14400 s 后保持末次观测；对照以末 10 个样本的均值延拓，对应空气温度 323.1965 K、空气水分浓度 0.049928 kg/kg。两种运行都记录为 206433 s，时间相对差异为 0。

现有检验将基准终点 206433 s 同时用作对照的最大运行窗口。求解器在窗口内未达标时也会返回窗口终点，而该检验报告未另存“是否实际触发阈值”的标志。因此相同返回时间只能说明这份有限窗口检查没有分辨出差异，不能单凭它证明两个连续交越时间完全相同。后续应延长对照上限并保存终点最大含水率和触发标志，再比较独立检测的交越时刻。正文保留已运行数据，不扩大其证据含义。

6.4 综合情景差异与未识别因素

固定半径、附录 3 物性的达标时间为 57.3425 h；实测收缩、附录 4 物性的达标时间为 50.8572 h，两者相差约 6.4853 h，相对缩短 11.31%。这是两项设定同时改变后的结果，不能识别几何和物性的单独贡献。半径缩小会改变扩散距离，但扩散系数也发生变化，两者还可能存在交互作用。

要定量区分因素，应补齐“收缩加附录 3”和“固定半径加附录 4”两种长程组合，并采用一致的初值、边界及阈值检测方法。现有证据只有两种主情景和一个 3600 s 的恒半径退化对照，不构成完整的四组合析因实验，所以不提供另外两种干燥时间或独立贡献率。

7 模型评价与推广

7.1 模型优点与结果的适用范围

本文把题目的输出位置、采样间隔和全域终点条件放在同一个径向数值框架内处理。圆柱壳层的面积、体积直接进入有限体积式，界面系数采用调和平均，边界串联半单元与对流阻力，使几何、内部传输和外部交换关系具有清晰的对应。三对角求解避免构造稠密矩阵，适于长时间逐秒积分 [1]。

四问之间的计算关系也得到明确区分：问题一与问题二各自从初始场出发，问题二的短期表格和问题三的长期终点使用同一经验物性，问题四则明确更换几何和物性。这样的组织方式能够避免把不同模型的末态拼接起来，并帮助判断某个结果差异来自哪一类设定。

预热阶段结果显示中心尚湿而表层已明显失水，全过程结果进一步显示温度先趋于平衡、含水率后趋于达标。因此工程判断应关注全域最大含水率，而不只看表面状态或环境温度。两种主情景的 57.3425 h 与 50.8572 h 是已声明边界和物性下的数值预测，可用于比较给定情景，不能直接作为所有药材品种的工艺常数。

7.2 模型局限与改进顺序

首先，宏观扩散近似没有分离多相迁移，也未显式求解蒸发潜热和干燥应力。空气与药材浓度的等效边界闭合缺少吸附平衡关系支持，实测标定后才适合扩大应用范围 [3, 5]。一维径向假设对中段较合理，但对两端区域和短粗物体需要增加轴向分辨率。

其次，物质坐标模型采用均匀径向收缩，未解变形本构、干骨架质量密度及孔隙结构。恒半径退化一致并不能替代动态验证。要比较几何收缩的独立作用，应先完成第 6 章所述组合计算，再用独立内部含水率观测检验结果，而不是把输入半径曲线与自身拟合的一致性当作预测准确度。

最后，当前时间推进每步固定更新两次物性，没有记录每步非线性残差；远期边界对照也存在终点窗口限制。这些问题决定了改进优先级：先增加触发标志和迭代残差记录，再进行长程网格、步长及迭代次数比较，随后开展边界扰动和物性不确定性分析。没有完成的实验不作为论文结果报告。

7.3 应用与推广建议

该框架可作为给定环境和几何条件下的离线模拟工具，用于查看内部梯度和比较候选运行情景。若要用于在线停止控制，需要额外的传感器校准、模型偏差评估与工艺安全裕量；现有数据不能直接计算药效保留率、能耗下降幅度或经济收益。对其他柱状食品或农产品，也应先重新确定物性、尺寸和边界关系，不能只替换名称便沿用参数 [4, 5]。

8 参考文献

[1] Patankar S V. Numerical Heat Transfer and Fluid Flow [M]. Hemisphere Publishing Corporation, 1980.

[2] Crank J. The Mathematics of Diffusion [M]. 2nd ed. Oxford: Clarendon Press, 1975.

[3] Whitaker S. Simultaneous heat, mass, and momentum transfer in porous media: A theory of drying [J]. Advances in Heat Transfer, 1977, 13: 119–203. DOI: 10.1016/S0065-2717(08)70223-5.

[4] Mujumdar A S, ed. Handbook of Industrial Drying [M]. 4th ed. CRC Press, 2014. DOI: 10.1201/b17208.

[5] Defraeye T. Advanced computational modelling for drying processes: A review [J]. Applied Energy, 2014, 131: 323–344. DOI: 10.1016/j.apenergy.2014.06.027.

[6] 2026 年高教社杯全国大学生数学建模竞赛. A 题: 药材的烘干问题及附件 [Z]. 本项目提供的赛题材料, 2026.

附录

附录 A：数据与求解复现说明

正文计算来自 `paper_output/code/modeling/` 下的 `q1_model.py` 至 `q4_model.py`，共同调用 `core_solver.py`。在仓库根目录、满足 `requirements.txt` 依赖的 Python 环境中运行 `python paper_output/code/modeling/run_modeling.py` 可生成模型输出。运行清单 `paper_output/results/run_manifest.json` 记录脚本、输入、输出与退出码，复算时应核对其中的文件摘要。

初始场为 301.15 K 和 2.55 kg/kg，基准空间网格为 80 单元、时间步长为 1 s、每步物性更新两次。环境和半径分别读取 `paper_output/data_cleaned/a_environment.csv`、`a_radius.csv`，采用分段线性插值。三对角算法的实际实现是 `core_solver.py` 中的 `solve_thomas`；本文不另列与实际程序脱节的示意代码。

`result1.xlsx` 含温度与水分浓度两表，从 1 至 1800 s 逐秒记录；`result2.xlsx` 同样有两表，从 1 s 延续至 206433 s，表 3、表 4 仅是前 3 h 的展示切片。`result3.xlsx` 按 60 s 保存固定半径含水率并额外保存 206433 s 终点；`result4.xlsx` 按 60 s 保存收缩域含水率并额外保存 183086 s 终点，最后一列为真实表面。四个文件均位于 `paper_output/tables/`。域外空值表示几何上不适用，不作为数值失败，也不填补为零。

附录 B：正文表格与证据文件对应

表 4-2、表 4-3 分别整理自 `table_data_profile_a_environment`、`table_data_profile_a_radius` 对应的 CSV。本文将重复单位、类型字段和开尔文转换列合并为中文表头，保留样本数、缺失数和统计量含义。

表 5-1 至表 5-6 依次对应 `table1` 至 `table6`。对应文件为 `table_aq1_temperature.csv`、`table_aq1_moisture.csv`、`table_aq2_temperature.csv`、`table_aq2_moisture.csv`、`table_aq3_drying_moisture.csv`、`table_aq4_shrinkage_moisture.csv`，均位于 `paper_output/tables/`。正文保留既有计算值，只统一编号、图题和解释口径。六幅图的原始图片路径已在源稿图像链接中注明。

附录 C：数值检验的追溯

数值对照来自 `paper_output/qa/model_validation_checks.json`，生成程序为 `paper_output/code/visualization/run_numerical_validation.py`。其中空间加密为 80 与 160 单元对照，时间加密为 1 与 0.5 s 对照；恒半径退化使用附录 4 物性运行 3600 s。环境延拓对照只覆盖末 10 点均值一种情景，并受基准终点窗口限制。

检验契约中的通过标志采用 1 (boolean) 表示，空间加密、时间加密、边界延拓检查和恒半径退化检查的记录值均为 1。边界相对差记录为 0 (fraction)，问题一总脱水比例记录为 0.063924 (fraction)。这些字段属于可追溯记录；正文用自然语言说明它们所支持的结论及限制，不将机器标志当成额外实验结果。

附录 D：完整可运行源程序

以下按文件列出本次求解与图表、数值检验所用的完整源程序；文件之间的相对导入关系保持不变。运行环境依赖列于最后，运行时在仓库根目录执行附录 A 给出的命令。

D.1 paper_output/code/modeling/core_solver.py

```
# Core solver module for radial finite volume method and Thomas algorithm
from __future__ import annotations

import numba
import numpy as np
import pandas as pd
from pathlib import Path

@numba.njit
def solve_thomas(a: np.ndarray, b: np.ndarray, c: np.ndarray, d: np.ndarray) ->
np.ndarray:
    n = len(d)
    cp = np.zeros(n, dtype=np.float64)
    dp = np.zeros(n, dtype=np.float64)
    cp[0] = c[0] / b[0]
    dp[0] = d[0] / b[0]
    for i in range(1, n):
        denom = b[i] - a[i] * cp[i - 1]
        cp[i] = c[i] / denom if i < n - 1 else 0.0
        dp[i] = (d[i] - a[i] * dp[i - 1]) / denom
    x = np.zeros(n, dtype=np.float64)
    x[-1] = dp[-1]
    for i in range(n - 2, -1, -1):
        x[i] = dp[i] - cp[i] * x[i + 1]
    return x

@numba.njit
def interp_env(t: float, t_env: np.ndarray, T_env: np.ndarray, C_env: np.ndarray) ->
tuple[float, float]:
    if t <= 0.0:
        return T_env[0], C_env[0]
    if t >= t_env[-1]:
        return T_env[-1], C_env[-1]
    idx = int(t / 60.0)
    if idx >= len(t_env) - 1:
```

```

        return T_env[-1], C_env[-1]
    dt_seg = t_env[idx + 1] - t_env[idx]
    frac = (t - t_env[idx]) / dt_seg if dt_seg > 0.0 else 0.0
    T_air = T_env[idx] + frac * (T_env[idx + 1] - T_env[idx])
    C_e = C_env[idx] + frac * (C_env[idx + 1] - C_env[idx])
    return T_air, C_e

@numba.njit
def interp_radius(t: float, t_rad: np.ndarray, R_rad: np.ndarray) -> float:
    if t <= 0.0:
        return R_rad[0]
    if t >= t_rad[-1]:
        return R_rad[-1]
    idx = int(t / 1800.0)
    if idx >= len(t_rad) - 1:
        return R_rad[-1]
    dt_seg = t_rad[idx + 1] - t_rad[idx]
    frac = (t - t_rad[idx]) / dt_seg if dt_seg > 0.0 else 0.0
    return R_rad[idx] + frac * (R_rad[idx + 1] - R_rad[idx])

@numba.njit
def simulate_fvm_fixed(
    t_env: np.ndarray,
    T_env: np.ndarray,
    C_env: np.ndarray,
    total_seconds: int,
    formula_mode: int, # 1 Appx 2 (AQ1), 2 Appx 3 (AQ2/AQ3), 3 Appx 4 fixed-domain
    reference_only (AQ4)
    sample_every_s: int = 1,
    N: int = 80,
    R: float = 0.02,
    h: float = 25.0,
    hm: float = 8e-7,
    dt: float = 1.0,
    stop_at_cmax: float = -1.0,
) -> tuple[np.ndarray, np.ndarray, np.ndarray, float, int]:
    dr = R / N
    r_faces = np.linspace(0.0, R, N + 1)
    vols = np.pi * (r_faces[1:]**2 - r_faces[:-1]**2)
    areas = 2.0 * np.pi * r_faces

    T = np.full(N, 301.15, dtype=np.float64)
    C = np.full(N, 2.55, dtype=np.float64)

    max_records = total_seconds // sample_every_s + 2
    times_out = np.zeros(max_records, dtype=np.float64)
    T_history = np.zeros((max_records, N), dtype=np.float64)
    C_history = np.zeros((max_records, N), dtype=np.float64)

    rec_idx = 0

```

```

times_out[rec_idx] = 0.0
T_history[rec_idx] = T
C_history[rec_idx] = C
rec_idx += 1

D_face = np.zeros(N + 1, dtype=np.float64)
k_face = np.zeros(N + 1, dtype=np.float64)

a_C = np.zeros(N, dtype=np.float64)
b_C = np.zeros(N, dtype=np.float64)
c_C = np.zeros(N, dtype=np.float64)
d_C = np.zeros(N, dtype=np.float64)

a_T = np.zeros(N, dtype=np.float64)
b_T = np.zeros(N, dtype=np.float64)
c_T = np.zeros(N, dtype=np.float64)
d_T = np.zeros(N, dtype=np.float64)

t = 0.0
step = 0
actual_end_time = float(total_seconds)

while t < total_seconds:
    t_next = t + dt
    T_air, C_e = interp_env(t_next, t_env, T_env, C_env)

    C_old = C.copy()
    T_old = T.copy()
    C_iter = C.copy()
    T_iter = T.copy()

    for _it in range(2):
        if formula_mode == 1:
            rho = np.full(N, 820.0, dtype=np.float64)
            cp = np.full(N, 2600.0, dtype=np.float64)
            k = np.full(N, 0.36, dtype=np.float64)
            D = 7.0e-9 * np.exp(-0.89 / C_iter)
        elif formula_mode == 2:
            rho = 650.0 + 128.0 * C_iter
            cp = 1450.0 + 2736.0 * (C_iter / (C_iter + 1.0))
            k = 0.21 + 0.38 * (C_iter / (C_iter + 1.0))
            D = 2.4e-3 * np.exp(-0.45 / C_iter - 3850.0 / T_iter)
        else: # formula_mode=3: Appendix 4 fixed-domain reference
            rho = 760.0 + 90.0 * C_iter
            cp = 1850.0 + 2150.0 * (C_iter / (C_iter + 1.0))
            k = 0.12 + 0.20 * (C_iter / (C_iter + 1.0))
            D = 4.2e-4 * np.exp(-0.30 / C_iter - 3850.0 / T_iter)

    for i in range(1, N):
        D_face[i] = 2.0 * D[i - 1] * D[i] / (D[i - 1] + D[i])
        gamma_s = 1.0 / (1.0 / hm + 0.5 * dr / D[N - 1])

```

```

for i in range(N):
    d_C[i] = vols[i] * C_old[i] / dt

f1 = areas[1] * D_face[1] / dr
b_C[0] = vols[0] / dt + f1
c_C[0] = -f1
a_C[0] = 0.0

for i in range(1, N - 1):
    f1 = areas[i] * D_face[i] / dr
    fr = areas[i + 1] * D_face[i + 1] / dr
    a_C[i] = -f1
    b_C[i] = vols[i] / dt + f1 + fr
    c_C[i] = -fr

f1 = areas[N - 1] * D_face[N - 1] / dr
fr_s = areas[N] * gamma_s
a_C[N - 1] = -f1
b_C[N - 1] = vols[N - 1] / dt + f1 + fr_s
c_C[N - 1] = 0.0
d_C[N - 1] += fr_s * C_e

C_iter = solve_thomas(a_C, b_C, c_C, d_C)

for i in range(1, N):
    k_face[i] = 2.0 * k[i - 1] * k[i] / (k[i - 1] + k[i])
gamma_T_s = 1.0 / (1.0 / h + 0.5 * dr / k[N - 1])

for i in range(N):
    d_T[i] = vols[i] * rho[i] * cp[i] * T_old[i] / dt

f1_T = areas[1] * k_face[1] / dr
b_T[0] = vols[0] * rho[0] * cp[0] / dt + f1_T
c_T[0] = -f1_T
a_T[0] = 0.0

for i in range(1, N - 1):
    f1 = areas[i] * k_face[i] / dr
    fr = areas[i + 1] * k_face[i + 1] / dr
    a_T[i] = -f1
    b_T[i] = vols[i] * rho[i] * cp[i] / dt + f1 + fr
    c_T[i] = -fr

f1 = areas[N - 1] * k_face[N - 1] / dr
fr_s = areas[N] * gamma_T_s
a_T[N - 1] = -f1
b_T[N - 1] = vols[N - 1] * rho[N - 1] * cp[N - 1] / dt + f1 + fr_s
c_T[N - 1] = 0.0
d_T[N - 1] += fr_s * T_air

T_iter = solve_thomas(a_T, b_T, c_T, d_T)

```

```

C = C_iter
T = T_iter
t = t_next
step += 1

if step % sample_every_s == 0:
    times_out[rec_idx] = t
    T_history[rec_idx] = T
    C_history[rec_idx] = C
    rec_idx += 1

if stop_at_cmax > 0.0 and C.max() < stop_at_cmax:
    actual_end_time = t
    if step % sample_every_s != 0:
        times_out[rec_idx] = t
        T_history[rec_idx] = T
        C_history[rec_idx] = C
        rec_idx += 1
    break

return times_out[:rec_idx], T_history[:rec_idx], C_history[:rec_idx], actual_end_time,
rec_idx

```

```

@numba.njit
def simulate_fvm_moving(
    t_env: np.ndarray,
    T_env: np.ndarray,
    C_env: np.ndarray,
    t_rad: np.ndarray,
    R_rad: np.ndarray,
    total_seconds: int,
    sample_every_s: int = 60,
    N: int = 80,
    h: float = 25.0,
    hm: float = 8e-7,
    dt: float = 1.0,
    stop_at_cmax: float = 0.15,
) -> tuple[np.ndarray, np.ndarray, np.ndarray, np.ndarray, float, int]:
    dxi = 1.0 / N
    xi_faces = np.linspace(0.0, 1.0, N + 1)
    vols_xi = np.pi * (xi_faces[1:]**2 - xi_faces[:-1]**2)
    areas_xi = 2.0 * np.pi * xi_faces

    T = np.full(N, 301.15, dtype=np.float64)
    C = np.full(N, 2.55, dtype=np.float64)

    max_records = total_seconds // sample_every_s + 2
    times_out = np.zeros(max_records, dtype=np.float64)
    radius_out = np.zeros(max_records, dtype=np.float64)
    T_history = np.zeros((max_records, N), dtype=np.float64)
    C_history = np.zeros((max_records, N), dtype=np.float64)

```

```

rec_idx = 0
times_out[rec_idx] = 0.0
radius_out[rec_idx] = R_rad[0]
T_history[rec_idx] = T
C_history[rec_idx] = C
rec_idx += 1

D_face = np.zeros(N + 1, dtype=np.float64)
k_face = np.zeros(N + 1, dtype=np.float64)

a_C = np.zeros(N, dtype=np.float64)
b_C = np.zeros(N, dtype=np.float64)
c_C = np.zeros(N, dtype=np.float64)
d_C = np.zeros(N, dtype=np.float64)

a_T = np.zeros(N, dtype=np.float64)
b_T = np.zeros(N, dtype=np.float64)
c_T = np.zeros(N, dtype=np.float64)
d_T = np.zeros(N, dtype=np.float64)

t = 0.0
step = 0
actual_end_time = float(total_seconds)

while t < total_seconds:
    t_next = t + dt
    T_air, C_e = interp_env(t_next, t_env, T_env, C_env)
    R_curr = interp_radius(t_next, t_rad, R_rad)

    C_old = C.copy()
    T_old = T.copy()
    C_iter = C.copy()
    T_iter = T.copy()

    for _it in range(2):
        rho = 760.0 + 90.0 * C_iter
        cp = 1850.0 + 2150.0 * (C_iter / (C_iter + 1.0))
        k = 0.12 + 0.20 * (C_iter / (C_iter + 1.0))
        D = 4.2e-4 * np.exp(-0.30 / C_iter - 3850.0 / T_iter)

        D_eff = D / (R_curr * R_curr)
        for i in range(1, N):
            D_face[i] = 2.0 * D_eff[i - 1] * D_eff[i] / (D_eff[i - 1] + D_eff[i])
            gamma_s_eff = 1.0 / ((0.5 * dxi * R_curr) / D[N - 1] + 1.0 / hm) / R_curr

        for i in range(N):
            d_C[i] = vols_xi[i] * C_old[i] / dt

        f1 = areas_xi[1] * D_face[1] / dxi
        b_C[0] = vols_xi[0] / dt + f1
        c_C[0] = -f1

```

```

a_C[0] = 0.0

for i in range(1, N - 1):
    fl = areas_xi[i] * D_face[i] / dxi
    fr = areas_xi[i + 1] * D_face[i + 1] / dxi
    a_C[i] = -fl
    b_C[i] = vols_xi[i] / dt + fl + fr
    c_C[i] = -fr

fl = areas_xi[N - 1] * D_face[N - 1] / dxi
fr_s = areas_xi[N] * gamma_s_eff
a_C[N - 1] = -fl
b_C[N - 1] = vols_xi[N - 1] / dt + fl + fr_s
c_C[N - 1] = 0.0
d_C[N - 1] += fr_s * C_e

C_iter = solve_thomas(a_C, b_C, c_C, d_C)

k_eff = k / (R_curr * R_curr)
for i in range(1, N):
    k_face[i] = 2.0 * k_eff[i - 1] * k_eff[i] / (k_eff[i - 1] + k_eff[i])
gamma_T_eff = 1.0 / ((0.5 * dxi * R_curr) / k[N - 1] + 1.0 / h) / R_curr

for i in range(N):
    d_T[i] = vols_xi[i] * rho[i] * cp[i] * T_old[i] / dt

fl_T = areas_xi[1] * k_face[1] / dxi
b_T[0] = vols_xi[0] * rho[0] * cp[0] / dt + fl_T
c_T[0] = -fl_T
a_T[0] = 0.0

for i in range(1, N - 1):
    fl = areas_xi[i] * k_face[i] / dxi
    fr = areas_xi[i + 1] * k_face[i + 1] / dxi
    a_T[i] = -fl
    b_T[i] = vols_xi[i] * rho[i] * cp[i] / dt + fl + fr
    c_T[i] = -fr

fl = areas_xi[N - 1] * k_face[N - 1] / dxi
fr_s = areas_xi[N] * gamma_T_eff
a_T[N - 1] = -fl
b_T[N - 1] = vols_xi[N - 1] * rho[N - 1] * cp[N - 1] / dt + fl + fr_s
c_T[N - 1] = 0.0
d_T[N - 1] += fr_s * T_air

T_iter = solve_thomas(a_T, b_T, c_T, d_T)

C = C_iter
T = T_iter
t = t_next
step += 1

```

```

        if step % sample_every_s == 0:
            times_out[rec_idx] = t
            radius_out[rec_idx] = R_curr
            T_history[rec_idx] = T
            C_history[rec_idx] = C
            rec_idx += 1

        if stop_at_cmax > 0.0 and C.max() < stop_at_cmax:
            actual_end_time = t
            if step % sample_every_s != 0:
                times_out[rec_idx] = t
                radius_out[rec_idx] = R_curr
                T_history[rec_idx] = T
                C_history[rec_idx] = C
                rec_idx += 1
            break

    return times_out[:rec_idx], radius_out[:rec_idx], T_history[:rec_idx],
    C_history[:rec_idx], actual_end_time, rec_idx

```

```

def sample_and_interpolate_fixed(
    times_raw: np.ndarray,
    T_raw: np.ndarray,
    C_raw: np.ndarray,
    N: int,
    R: float,
    r_target_cm: np.ndarray,
) -> tuple[np.ndarray, np.ndarray]:
    r_faces = np.linspace(0.0, R, N + 1)
    r_centers = 0.5 * (r_faces[:-1] + r_faces[1:])
    r_target_m = r_target_cm / 100.0
    n_steps = len(times_raw)
    n_radii = len(r_target_m)
    T_interp = np.zeros((n_steps, n_radii), dtype=np.float64)
    C_interp = np.zeros((n_steps, n_radii), dtype=np.float64)
    for s in range(n_steps):
        T_interp[s] = np.interp(r_target_m, r_centers, T_raw[s])
        C_interp[s] = np.interp(r_target_m, r_centers, C_raw[s])
    return T_interp, C_interp

```

```

def sample_and_interpolate_moving(
    times_raw: np.ndarray,
    radius_raw: np.ndarray,
    C_raw: np.ndarray,
    N: int,
    r_target_cm: np.ndarray,
) -> tuple[np.ndarray, np.ndarray]:
    xi_faces = np.linspace(0.0, 1.0, N + 1)
    xi_centers = 0.5 * (xi_faces[:-1] + xi_faces[1:])
    r_target_m = r_target_cm / 100.0

```

```

n_steps = len(times_raw)
n_radai = len(r_target_m)
C_interp = np.full((n_steps, n_radai), np.nan, dtype=np.float64)
C_surface = C_raw[:, -1].copy()
for s in range(n_steps):
    R_s = radius_raw[s]
    valid_mask = r_target_m <= R_s
    if np.any(valid_mask):
        xi_targets = r_target_m[valid_mask] / R_s
        C_interp[s, valid_mask] = np.interp(xi_targets, xi_centers, C_raw[s])
return C_interp, C_surface

```

D.2 paper_output/code/modeling/q1_model.py

```

# Generated by MathModel Skill scaffold generator.
from __future__ import annotations

import csv
import json
from pathlib import Path
import numpy as np
import pandas as pd
import openpyxl

from core_solver import simulate_fvm_fixed, sample_and_interpolate_fixed
from result_contract_io import (
    upsert_question_contracts,
    write_csv_table,
    table_entry,
    round_float,
    PROJECT_ROOT,
    OUTPUT_DIR,
    TABLES_DIR,
    RESULTS_DIR,
    DATA_CLEANED_DIR,
)

QUESTION = {
    'question_id': 'AQ1',
    'title': '预热平衡阶段温度与干基含水率',
    'task_type': '非线性扩散与热传导',
    'main_model': '隐式有限体积法 (FVM) + Picard 迭代 + Thomas 三对角求解',
}

def run_aq1():
    env_path = DATA_CLEANED_DIR / 'a_environment.csv'
    df_env = pd.read_csv(env_path)
    t_env = df_env['time_s'].values.astype(np.float64)
    T_env = df_env['temperature_K'].values.astype(np.float64)
    C_env = df_env['air_moisture_kgkg'].values.astype(np.float64)

    # 1. Run simulation for 1800 s with formula_mode=1 (Appendix 2)

```

```

times_raw, T_raw, C_raw, end_time, n_rec = simulate_fvm_fixed(
    t_env=t_env,
    T_env=T_env,
    C_env=C_env,
    total_seconds=1800,
    formula_mode=1,
    sample_every_s=1,
    N=80,
    R=0.02,
    h=25.0,
    hm=8e-7,
    dt=1.0,
)

# 2. Build Table 1 (Temperature) & Table 2 (Moisture)
times_tab = np.array([100, 300, 600, 900, 1200, 1500, 1800], dtype=np.float64)
r_tab_cm = np.array([0.0, 0.5, 1.0, 1.5, 2.0], dtype=np.float64)

# Interpolate to table points
T_tab, C_tab = sample_and_interpolate_fixed(times_tab, T_raw[times_tab.astype(int)],
C_raw[times_tab.astype(int)], N=80, R=0.02, r_target_cm=r_tab_cm)

# Convert Temperature to Celsius
T_tab_C = T_tab - 273.15

# Write Table 1 CSV
table1_rows = []
for i, t_val in enumerate(times_tab):
    row = {'时间/s': int(t_val)}
    for j, r_val in enumerate(r_tab_cm):
        row[f'{r_val:g} cm'] = f'{T_tab_C[i, j]:.4f}'
    table1_rows.append(row)
t1_path = TABLES_DIR / 'table_aq1_temperature.csv'
write_csv_table(t1_path, table1_rows)

# Write Table 2 CSV
table2_rows = []
for i, t_val in enumerate(times_tab):
    row = {'时间/s': int(t_val)}
    for j, r_val in enumerate(r_tab_cm):
        row[f'{r_val:g} cm'] = f'{C_tab[i, j]:.4f}'
    table2_rows.append(row)
t2_path = TABLES_DIR / 'table_aq1_moisture.csv'
write_csv_table(t2_path, table2_rows)

# 3. Build result1.xlsx (1800 s, 0.1 cm steps)
r_full_cm = np.arange(0.0, 2.0001, 0.1)
T_full, C_full = sample_and_interpolate_fixed(times_raw, T_raw, C_raw, N=80, R=0.02,
r_target_cm=r_full_cm)
T_full_C = T_full - 273.15

res1_path = TABLES_DIR / 'result1.xlsx'

```

```

wb = openpyxl.Workbook(write_only=True)
ws_T = wb.create_sheet('温度')
ws_C = wb.create_sheet('水分度')

header = ['时间\\到药材中心的距离'] + [round(r, 1) for r in r_full_cm]
ws_T.append(header)
ws_C.append(header)

# 1 to 1800 s
for s in range(1, 1801):
    row_T = [s] + [round(val, 4) for val in T_full_C[s]]
    row_C = [s] + [round(val, 4) for val in C_full[s]]
    ws_T.append(row_T)
    ws_C.append(row_C)
wb.save(res1_path)

# 4. Metrics & Contracts
metrics = [
    {'metric_name': 'aq1_final_center_temperature_C', 'metric_role': 'evaluation',
'value': round_float(T_full_C[1800, 0], 4), 'unit': '°C'},
    {'metric_name': 'aq1_final_surface_temperature_C', 'metric_role': 'evaluation',
'value': round_float(T_full_C[1800, -1], 4), 'unit': '°C'},
    {'metric_name': 'aq1_final_center_moisture', 'metric_role': 'evaluation', 'value':
round_float(C_full[1800, 0], 4), 'unit': 'kg/kg'},
    {'metric_name': 'aq1_final_surface_moisture', 'metric_role': 'evaluation',
'value': round_float(C_full[1800, -1], 4), 'unit': 'kg/kg'},
    {'metric_name': 'aq1_total_moisture_loss_fraction', 'metric_role': 'evaluation',
'value': round_float((2.55 - np.mean(C_full[1800]))) / 2.55, 6), 'unit': 'fraction'},
]

conclusions = [
    {
        'question_id': 'AQ1',
        'conclusion_text': f'在预热平衡阶段 (0-1800 s), 药材受烘房升温与对流传热驱动, 中心温度由 28.0000°C 上升至{T_full_C[1800, 0]:.4f}°C, 表面温度达到{T_full_C[1800, -1]:.4f}°C; 由于对流传质阻力与内部非线性水分扩散制约, 药材表面干基含水率由 2.5500 kg/kg 下降至{C_full[1800, -1]:.4f} kg/kg, 中心干基含水率为{C_full[1800, 0]:.4f} kg/kg, 体出著的内外梯度滞效。',
        'evidence_required': ['model_results.json', 'metrics.json',
'table_index.json'],
        'evidence_status': 'computed',
    }
]

tables = [
    table_entry(
        question_id='AQ1',
        table_id='table1',
        title='30 分钟内药材的温度 (表 1) ',
        purpose='展示预热阶段 100-1800 s 各径向截面的药材温度动态。',
        path=t1_path,
        status='computed',
    ),

```

```

        table_entry(
            question_id='AQ1',
            table_id='table2',
            title='30 分钟内药材的水分浓度（表 2）',
            purpose='展示预热阶段 100-1800 s 各径向截面的干基水分浓度分布。',
            path=t2_path,
            status='computed',
        ),
    ]

    outputs = [
        {'path': str(t1_path), 'type': 'csv'},
        {'path': str(t2_path), 'type': 'csv'},
        {'path': str(res1_path), 'type': 'xlsx'},
    ]

    summary = f'AQ1 预热平衡阶段数值求解完成：1800 s 时中心温度为{T_full_C[1800, 0]:.4f}°C，表面
温度为{T_full_C[1800, -1]:.4f}°C；中心水分浓度为{C_full[1800, 0]:.4f} kg/kg，表面水分浓度为
{C_full[1800, -1]:.4f} kg/kg。表 1、表 2 及 result1.xlsx 均已导出。'

    upsert_question_contracts(
        question=QUESTION,
        result_summary=summary,
        metrics=metrics,
        tables=tables,
        conclusions=conclusions,
        outputs=outputs,
        status='computed',
    )
    print('AQ1 finished successfully.')

if __name__ == '__main__':
    run_aq1()

```

D.3 paper_output/code/modeling/q2_model.py

```

# Generated by MathModel Skill scaffold generator.
from __future__ import annotations

import csv
import json
from pathlib import Path
import numpy as np
import pandas as pd
import openpyxl

from core_solver import simulate_fvm_fixed, sample_and_interpolate_fixed
from result_contract_io import (
    upsert_question_contracts,
    write_csv_table,
    table_entry,
    round_float,

```

```

PROJECT_ROOT,
OUTPUT_DIR,
TABLES_DIR,
RESULTS_DIR,
DATA_CLEANED_DIR,
)

QUESTION = {
    'question_id': 'AQ2',
    'title': '统一经验物性的全过程模型',
    'task_type': '热湿非线性耦合',
    'main_model': '附录3 全过程经验物性FVM模型 + Picard迭代 + Thomas三对角求解',
}

def run_aq2():
    env_path = DATA_CLEANED_DIR / 'a_environment.csv'
    df_env = pd.read_csv(env_path)
    t_env = df_env['time_s'].values.astype(np.float64)
    T_env = df_env['temperature_K'].values.astype(np.float64)
    C_env = df_env['air_moisture_kgkg'].values.astype(np.float64)

    # Appendix 3 applies from t=0 until drying completes. Tables 3/4
    # show only the first 3 h; result2.xlsx must retain every second.
    times_raw, T_raw, C_raw, end_time, n_rec = simulate_fvm_fixed(
        t_env=t_env,
        T_env=T_env,
        C_env=C_env,
        total_seconds=259200,
        formula_mode=2,
        sample_every_s=1,
        N=80,
        R=0.02,
        h=25.0,
        hm=8e-7,
        dt=1.0,
        stop_at_cmax=0.15,
    )

    # 2. Build Table 3 (Temperature) & Table 4 (Moisture)
    # 0.5 to 3.0 h every 0.5 h -> [1800, 3600, 5400, 7200, 9000, 10800] s
    times_tab_s = np.array([1800, 3600, 5400, 7200, 9000, 10800], dtype=np.float64)
    times_tab_h = times_tab_s / 3600.0
    r_tab_cm = np.array([0.0, 0.5, 1.0, 1.5, 2.0], dtype=np.float64)

    T_tab, C_tab = sample_and_interpolate_fixed(times_tab_s,
        T_raw[times_tab_s.astype(int)], C_raw[times_tab_s.astype(int)], N=80, R=0.02,
        r_target_cm=r_tab_cm)
    T_tab_C = T_tab - 273.15

    # Write Table 3 CSV
    table3_rows = []
    for i, t_h in enumerate(times_tab_h):

```

```

        row = {'时间/h': f'{t_h:.1f}'}
        for j, r_val in enumerate(r_tab_cm):
            row[f'{r_val:g} cm'] = f'{T_tab_C[i, j]:.4f}'
        table3_rows.append(row)
t3_path = TABLES_DIR / 'table_aq2_temperature.csv'
write_csv_table(t3_path, table3_rows)

# Write Table 4 CSV
table4_rows = []
for i, t_h in enumerate(times_tab_h):
    row = {'时间/h': f'{t_h:.1f}'}
    for j, r_val in enumerate(r_tab_cm):
        row[f'{r_val:g} cm'] = f'{C_tab[i, j]:.4f}'
    table4_rows.append(row)
t4_path = TABLES_DIR / 'table_aq2_moisture.csv'
write_csv_table(t4_path, table4_rows)

# 3. Build result2.xlsx (every second to the terminal event, 0.1 cm steps)
r_full_cm = np.arange(0.0, 2.0001, 0.1)
T_full, C_full = sample_and_interpolate_fixed(times_raw, T_raw, C_raw, N=80, R=0.02,
r_target_cm=r_full_cm)
T_full_C = T_full - 273.15

res2_path = TABLES_DIR / 'result2.xlsx'
wb = openpyxl.Workbook(write_only=True)
ws_T = wb.create_sheet('温度')
ws_C = wb.create_sheet('水分浓度')

header = ['时间\\到药材中心的距离'] + [round(r, 1) for r in r_full_cm]
ws_T.append(header)
ws_C.append(header)

for s in range(1, len(times_raw)):
    row_T = [s] + [round(val, 4) for val in T_full_C[s]]
    row_C = [s] + [round(val, 4) for val in C_full[s]]
    ws_T.append(row_T)
    ws_C.append(row_C)
wb.save(res2_path)

# 4. Metrics & Contracts
metrics = [
    {'metric_name': 'aq2_3h_center_temperature_C', 'metric_role': 'evaluation',
'value': round_float(T_full_C[10800, 0], 4), 'unit': '°C'},
    {'metric_name': 'aq2_3h_surface_temperature_C', 'metric_role': 'evaluation',
'value': round_float(T_full_C[10800, -1], 4), 'unit': '°C'},
    {'metric_name': 'aq2_3h_center_moisture', 'metric_role': 'evaluation', 'value':
round_float(C_full[10800, 0], 4), 'unit': 'kg/kg'},
    {'metric_name': 'aq2_3h_surface_moisture', 'metric_role': 'evaluation', 'value':
round_float(C_full[10800, -1], 4), 'unit': 'kg/kg'},
    {'metric_name': 'aq2_1800s_surface_temperature_C', 'metric_role': 'comparison',
'value': round_float(T_full_C[1800, -1], 4), 'unit': '°C'},
    {'metric_name': 'aq2_1800s_surface_moisture', 'metric_role': 'comparison',

```

```

'value': round_float(C_full[1800, -1], 4), 'unit': 'kg/kg'},
]

conclusions = [
    {
        'question_id': 'AQ2',
        'conclusion_text': f'采用附录3全过程统一经验物性公式自t=0连续求解3 h (10800 s) :
药材升温迅速并在3 h内达到中心{T_full_C[10800, 0]:.4f}°C、表面{T_full_C[10800, -1]:.4f}°C, 整体近烘房温度 ( 50.2°C) ; 水分浓度在3 h末中心降至{C_full[10800, 0]:.4f} kg/kg, 表面降至
{C_full[10800, -1]:.4f} kg/kg。相于AQ1常物性模型, 附录3考虑温度和含水率对扩散及热传导的非线性强化, 物性场平滑自洽, 无须拼接即可覆盖预热与恒温全过程。',
        'evidence_required': ['model_results.json', 'metrics.json',
'table_index.json'],
        'evidence_status': 'computed',
    }
]

tables = [
    table_entry(
        question_id='AQ2',
        table_id='table3',
        title='3小时内药材的温度 (表3) ',
        purpose='展示全过程模型在0.5-3.0 h各半小的温度径向分布。 ',
        path=t3_path,
        status='computed',
    ),
    table_entry(
        question_id='AQ2',
        table_id='table4',
        title='3小时内药材的水分浓度 (表4) ',
        purpose='展示全过程模型在0.5-3.0 h各半小时的干基水分浓度径向分布。',
        path=t4_path,
        status='computed',
    ),
]

outputs = [
    {'path': str(t3_path), 'type': 'csv'},
    {'path': str(t4_path), 'type': 'csv'},
    {'path': str(res2_path), 'type': 'xlsx'},
]

summary = f'AQ2全过程模型求解至达标时刻{end_time:.0f} s : 3 h时中心温度为{T_full_C[10800, 0]:.4f}°C, 表面温度为{T_full_C[10800, -1]:.4f}°C ; 中心水分浓度为{C_full[10800, 0]:.4f} kg/kg, 表面水分浓度为{C_full[10800, -1]:.4f} kg/kg。表3、表4保留前3 h, result2.xlsx包含1 s至达标时刻的逐秒温度和含水率。'

upsert_question_contracts(
    question=QUESTION,
    result_summary=summary,
    metrics=metrics,
    tables=tables,

```

```

        conclusions=conclusions,
        outputs=outputs,
        status='computed',
    )
    print('AQ2 finished successfully.')

if __name__ == '__main__':
    run_aq2()

```

D. 4 paper_output/code/modeling/q3_model.py

```

# Generated by MathModel Skill scaffold generator.
from __future__ import annotations

import csv
import json
from pathlib import Path
import numpy as np
import pandas as pd
import openpyxl

from core_solver import simulate_fvm_fixed, sample_and_interpolate_fixed
from result_contract_io import (
    upsert_question_contracts,
    write_csv_table,
    table_entry,
    round_float,
    PROJECT_ROOT,
    OUTPUT_DIR,
    TABLES_DIR,
    RESULTS_DIR,
    DATA_CLEANED_DIR,
)

QUESTION = {
    'question_id': 'AQ3',
    'title': '固定半径下全域达标时长',
    'task_type': '阈值事件检测',
    'main_model': '烘干终点全域极值检测 + 截断逼近算法',
}

def run_aq3():
    env_path = DATA_CLEANED_DIR / 'a_environment.csv'
    df_env = pd.read_csv(env_path)
    t_env = df_env['time_s'].values.astype(np.float64)
    T_env = df_env['temperature_K'].values.astype(np.float64)
    C_env = df_env['air_moisture_kgkg'].values.astype(np.float64)

    # 1. Run simulation until max(C) < 0.15 kg/kg (sampling every 60 s)
    # Maximum horizon 72 h = 259200 s
    times_raw, T_raw, C_raw, end_time, n_rec = simulate_fvm_fixed(
        t_env=t_env,

```

```

    T_env=T_env,
    C_env=C_env,
    total_seconds=259200,
    formula_mode=2,
    sample_every_s=60,
    N=80,
    R=0.02,
    h=25.0,
    hm=8e-7,
    dt=1.0,
    stop_at_cmax=0.15,
)

t_end_s = end_time
t_end_h = t_end_s / 3600.0

# 2. Build Table 5 (Moisture every 6 h + drying end time)
# Target radii: 0, 0.5, 1.0, 1.5, 2.0 cm
r_tab_cm = np.array([0.0, 0.5, 1.0, 1.5, 2.0], dtype=np.float64)

# 6 h intervals in seconds: 6h=21600s, 12h=43200s, ... up to t_end_s
max_6h = int(t_end_s // 21600)
times_6h_s = [i * 21600 for i in range(1, max_6h + 1)]
if t_end_s not in times_6h_s:
    times_tab_s = np.array(times_6h_s + [t_end_s], dtype=np.float64)
else:
    times_tab_s = np.array(times_6h_s, dtype=np.float64)

# Find the matching indices in times_raw
tab_indices = []
for ts in times_tab_s:
    idx = np.argmin(np.abs(times_raw - ts))
    tab_indices.append(idx)
tab_indices = np.array(tab_indices)

T_tab, C_tab = sample_and_interpolate_fixed(
    times_tab_s,
    T_raw[tab_indices],
    C_raw[tab_indices],
    N=80,
    R=0.02,
    r_target_cm=r_tab_cm,
)

table5_rows = []
for i, ts in enumerate(times_tab_s):
    if i == len(times_tab_s) - 1 and abs(ts - t_end_s) < 1e-3:
        time_label = f'烘干结束时间 ({t_end_h:.4f} h)'
    else:
        time_label = f'{ts / 3600.0:.1f}'
    row = {'时间/h': time_label}
    for j, r_val in enumerate(r_tab_cm):

```

```

        row[f'{r_val:g} cm'] = f'{C_tab[i, j]:.4f}'
    table5_rows.append(row)

t5_path = TABLES_DIR / 'table_aq3_drying_moisture.csv'
write_csv_table(t5_path, table5_rows)

# 3. Build result3.xlsx (every 60 s + end time, 0.1 cm steps)
r_full_cm = np.arange(0.0, 2.0001, 0.1)
T_full, C_full = sample_and_interpolate_fixed(
    times_raw,
    T_raw,
    C_raw,
    N=80,
    R=0.02,
    r_target_cm=r_full_cm,
)

res3_path = TABLES_DIR / 'result3.xlsx'
wb = openpyxl.Workbook(write_only=True)
ws = wb.create_sheet('Sheet1')

header = ['时间\\到药材中心的距离'] + [round(r, 1) for r in r_full_cm]
ws.append(header)

for i in range(1, len(times_raw)):
    t_sec = int(round(times_raw[i]))
    row = [t_sec] + [round(val, 4) for val in C_full[i]]
    ws.append(row)
wb.save(res3_path)

# 4. Metrics & Contracts
metrics = [
    {'metric_name': 'aq3_drying_time_s', 'metric_role': 'evaluation', 'value':
round_float(t_end_s, 2), 'unit': 's'},
    {'metric_name': 'aq3_drying_time_h', 'metric_role': 'evaluation', 'value':
round_float(t_end_h, 4), 'unit': 'h'},
    {'metric_name': 'aq3_final_max_moisture', 'metric_role': 'evaluation', 'value':
round_float(np.max(C_full[-1]), 4), 'unit': 'kg/kg'},
    {'metric_name': 'aq3_final_min_moisture', 'metric_role': 'evaluation', 'value':
round_float(np.min(C_full[-1]), 4), 'unit': 'kg/kg'},
    {'metric_name': 'aq3_final_center_moisture', 'metric_role': 'evaluation', 'value':
round_float(C_full[-1, 0], 4), 'unit': 'kg/kg'},
    {'metric_name': 'aq3_final_surface_moisture', 'metric_role': 'evaluation',
'value': round_float(C_full[-1, -1], 4), 'unit': 'kg/kg'},
]

conclusions = [
    {
        'question_id': 'AQ3',
        'conclusion_text': f'在固定几何尺寸（半径 2 cm）下，严格以药材全域最大含水率低于 0.15
kg/kg 作为烘干完成判定准则，模型精确确定烘干所需时间为{t_end_h:.4f}小时（即{t_end_s:.1f}秒，约
57.34 小时，处于 2-3 天工艺区间内）。烘干结束时刻中心含水率为{C_full[-1, 0]:.4f} kg/kg，表面含水率

```

为{C_full[-1, -1]:.4f} kg/kg, 全域最大含水率达到达标临界值 0.1500 kg/kg。表 5 及 result3.xlsx 均已精确导出。',

```

        'evidence_required': ['model_results.json', 'metrics.json',
'example_index.json'],
        'evidence_status': 'computed',
    }
]

tables = [
    table_entry(
        question_id='AQ3',
        table_id='table5',
        title='药材烘干过程的水分浓度 (表 5) ',
        purpose='展示固定半径下每隔 6 小时及烘干结束时刻各截面含水率衰变历程。',
        path=t5_path,
        status='computed',
    ),
]

outputs = [
    {'path': str(t5_path), 'type': 'csv'},
    {'path': str(res3_path), 'type': 'xlsx'},
]

summary = f'AQ3 固定半径烘干时长确定完成：烘干达标时长为{t_end_h:.4f} h ({t_end_s:.1f} s)；
终点中心水分浓度为{C_full[-1, 0]:.4f} kg/kg。表面为{C_full[-1, -1]:.4f} kg/kg。表 5 与
result3.xlsx 已导出。'

upsert_question_contracts(
    question=QUESTION,
    result_summary=summary,
    metrics=metrics,
    tables=tables,
    conclusions=conclusions,
    outputs=outputs,
    status='computed',
)
print(f'AQ3 finished successfully: t_end = {t_end_h:.4f} h ({t_end_s:.1f} s).')

if __name__ == '__main__':
    run_aq3()

```

D.5 paper_output/code/modeling/q4_model.py

```

# Generated by MathModel Skill scaffold generator.
from __future__ import annotations

import csv
import json
from pathlib import Path
import numpy as np
import pandas as pd

```

```

import openpyxl

from core_solver import simulate_fvm_moving, sample_and_interpolate_moving
from result_contract_io import (
    upsert_question_contracts,
    write_csv_table,
    table_entry,
    round_float,
    PROJECT_ROOT,
    OUTPUT_DIR,
    TABLES_DIR,
    RESULTS_DIR,
    DATA_CLEANED_DIR,
)

QUESTION = {
    'question_id': 'AQ4',
    'title': '收缩域中的烘干时长',
    'task_type': '移动边界传热传质',
    'main_model': '物质坐标系(xi)移网格 FVM + 附录4公式 + 截断逼近算法',
}

def run_aq4():
    env_path = DATA_CLEANED_DIR / 'a_environment.csv'
    df_env = pd.read_csv(env_path)
    t_env = df_env['time_s'].values.astype(np.float64)
    T_env = df_env['temperature_K'].values.astype(np.float64)
    C_env = df_env['air_moisture_kgkg'].values.astype(np.float64)

    rad_path = DATA_CLEANED_DIR / 'a_radius.csv'
    df_rad = pd.read_csv(rad_path)
    t_rad = df_rad['time_s'].values.astype(np.float64)
    R_rad = df_rad['radius_m'].values.astype(np.float64)

    # 1. Run simulation until max(C) < 0.15 kg/kg (sampling every 60 s)
    times_raw, radius_raw, T_raw, C_raw, end_time, n_rec = simulate_fvm_moving(
        t_env=t_env,
        T_env=T_env,
        C_env=C_env,
        t_rad=t_rad,
        R_rad=R_rad,
        total_seconds=259200,
        sample_every_s=60,
        N=80,
        h=25.0,
        hm=8e-7,
        dt=1.0,
        stop_at_cmax=0.15,
    )

    t_end_s = end_time
    t_end_h = t_end_s / 3600.0

```

```

# 2. Build Table 6 (Moisture every 6 h + drying end time)
# Target radii in domain: 0, 0.5, 1.0, 1.5 cm (and 2.0 cm if applicable, plus actual
surface)
r_tab_cm = np.array([0.0, 0.5, 1.0, 1.5, 2.0], dtype=np.float64)

max_6h = int(t_end_s // 21600)
times_6h_s = [i * 21600 for i in range(1, max_6h + 1)]
if t_end_s not in times_6h_s:
    times_tab_s = np.array(times_6h_s + [t_end_s], dtype=np.float64)
else:
    times_tab_s = np.array(times_6h_s, dtype=np.float64)

tab_indices = []
for ts in times_tab_s:
    idx = np.argmin(np.abs(times_raw - ts))
    tab_indices.append(idx)
tab_indices = np.array(tab_indices)

C_tab_interp, C_tab_surf = sample_and_interpolate_moving(
    times_tab_s,
    radius_raw[tab_indices],
    C_raw[tab_indices],
    N=80,
    r_target_cm=r_tab_cm,
)

table6_rows = []
for i, ts in enumerate(times_tab_s):
    if i == len(times_tab_s) - 1 and abs(ts - t_end_s) < 1e-3:
        time_label = f'烘干结束时间 ({t_end_h:.4f} h)'
    else:
        time_label = f'{ts / 3600.0:.1f}'
    row = {'时间/h': time_label}
    for j, r_val in enumerate(r_tab_cm):
        val = C_tab_interp[i, j]
        row[f'{r_val:g} cm'] = f'{val:.4f}' if not np.isnan(val) else ''
    row['药材表面'] = f'{C_tab_surf[i]:.4f}'
    row['当前半径/cm'] = f'{radius_raw[tab_indices[i]] * 100.0:.3f}'
    table6_rows.append(row)

t6_path = TABLES_DIR / 'table_aq4_shrinkage_moisture.csv'
write_csv_table(t6_path, table6_rows)

# 3. Build result4.xlsx (every 60 s + end time, 0.1 cm steps + surface)
r_full_cm = np.arange(0.0, 2.0001, 0.1)
C_full_interp, C_full_surf = sample_and_interpolate_moving(
    times_raw,
    radius_raw,
    C_raw,
    N=80,
    r_target_cm=r_full_cm,

```

```

)

res4_path = TABLES_DIR / 'result4.xlsx'
wb = openpyxl.Workbook(write_only=True)
ws = wb.create_sheet('Sheet1')

header = ['时间\\到材中心的距离 ' ] + [round(r, 1) for r in r_full_cm] + ['药材表面']
ws.append(header)

for i in range(1, len(times_raw)):
    t_sec = int(round(times_raw[i]))
    row = [t_sec]
    for val in C_full_interp[i]:
        row.append(round(val, 4) if not np.isnan(val) else None)
    row.append(round(C_full_surf[i], 4))
    ws.append(row)
wb.save(res4_path)

# 4. Metrics & Contracts
metrics = [
    {'metric_name': 'aq4_drying_time_s', 'metric_role': 'evaluation', 'value':
round_float(t_end_s, 2), 'unit': 's'},
    {'metric_name': 'aq4_drying_time_h', 'metric_role': 'evaluation', 'value':
round_float(t_end_h, 4), 'unit': 'h'},
    {'metric_name': 'aq4_final_radius_cm', 'metric_role': 'evaluation', 'value':
round_float(radius_raw[-1] * 100.0, 4), 'unit': 'cm'},
    {'metric_name': 'aq4_final_max_moisture', 'metric_role': 'evaluation', 'value':
round_float(np.max(C_raw[-1]), 4), 'unit': 'kg/kg'},
    {'metric_name': 'aq4_final_min_moisture', 'metric_role': 'evaluation', 'value':
round_float(np.min(C_raw[-1]), 4), 'unit': 'kg/kg'},
    {'metric_name': 'aq4_final_center_moisture', 'metric_role': 'evaluation', 'value':
round_float(C_raw[-1, 0], 4), 'unit': 'kg/kg'},
    {'metric_name': 'aq4_final_surface_moisture', 'metric_role': 'evaluation',
'value': round_float(C_full_surf[-1], 4), 'unit': 'kg/kg'},
    {'metric_name': 'aq4_time_reduction_vs_aq3_percent', 'metric_role': 'comparison',
'value': round_float((57.3425 - t_end_h) / 57.3425 * 100.0, 2), 'unit': '%'},
]

conclusions = [
    {
        'question_id': 'AQ4',
        'conclusion_text': f'在考虑径向动态收缩及附录4独立经验物性组条件下，药材烘干所需时间显著缩短至{t_end_h:.4f}小时（即{t_end_s:.1f}秒，约50.86小时），相比固定半径模型（AQ3的57.3425小时）缩短了约11.31%。烘干结束时药材收缩至半径{radius_raw[-1] * 100.0:.3f} cm，中心含水率为{C_raw[-1, 0]:.4f} kg/kg，表面含水率为{C_full_surf[-1]:.4f} kg/kg，全域最大含水率达到临界阈值0.1500 kg/kg。几何尺寸减小大幅缩短了内部水分析出路径，显著提升了烘干动力学速率。表6及result4.xlsx均已导出。',
        'evidence_required': ['model_results.json', 'metrics.json',
'table_index.json'],
        'evidence_status': 'computed',
    }
]

```

```

tables = [
    table_entry(
        question_id='AQ4',
        table_id='table6',
        title='药材烘干过程的水分浓度（表6）',
        purpose='展示收域中每隔6小时及烘干结束时刻各物理截面及真实表面的含水率分布。',
        path=t6_path,
        status='computed',
    ),
]

outputs = [
    {'path': str(t6_path), 'type': 'csv'},
    {'path': str(res4_path), 'type': 'xlsx'},
]

summary = f'AQ4 收缩域烘干时长求解完成：烘干达标时长为{t_end_h:.4f} h ({t_end_s:.1f} s)；
点半径为{radius_raw[-1] * 100.0:.3f} cm，中心含水率为{C_raw[-1, 0]:.4f} kg/kg，表面为
{C_full_surf[-1]:.4f} kg/kg。表6与result4.xlsx已导出。'

upsert_question_contracts(
    question=QUESTION,
    result_summary=summary,
    metrics=metrics,
    tables=tables,
    conclusions=conclusions,
    outputs=outputs,
    status='computed',
)

print(f'AQ4 finished successfully: t_end = {t_end_h:.4f} h ({t_end_s:.1f} s).')

if __name__ == '__main__':
    run_aq4()

```

D. 6 paper_output/code/modeling/result_contract_io.py

```

# Generated by MathModel Skill scaffold generator.
from __future__ import annotations

import csv
import hashlib
import json
import math
import sys
from datetime import datetime
from pathlib import Path
from typing import Any

import numpy as np
import pandas as pd

```

```

THIS_FILE = Path(__file__).resolve()

def detect_project_root() -> Path:
    for parent in THIS_FILE.parents:
        if parent.name == "paper_output":
            return parent.parent
    return Path.cwd().resolve()

PROJECT_ROOT = detect_project_root()
OUTPUT_DIR = PROJECT_ROOT / "paper_output"
RESULTS_DIR = OUTPUT_DIR / "results"
TABLES_DIR = OUTPUT_DIR / "tables"
DATA_CLEANED_DIR = OUTPUT_DIR / "data_cleaned"

def now() -> str:
    return datetime.now().isoformat(timespec="seconds")

def rel(path: Path) -> str:
    try:
        return path.resolve().relative_to(PROJECT_ROOT).as_posix()
    except Exception:
        return str(path).replace("\\", "/")

def contract_path(value: object) -> str:
    """Return a project-relative path for persisted contract fields."""
    if not value:
        return ""
    path = Path(str(value))
    return rel(path if path.is_absolute() else PROJECT_ROOT / path)

def normalize_outputs(outputs: list[dict[str, Any]]) -> list[dict[str, Any]]:
    normalized = []
    for item in outputs:
        if not isinstance(item, dict):
            continue
        copy = dict(item)
        if copy.get("path"):
            copy["path"] = contract_path(copy["path"])
        normalized.append(copy)
    return normalized

def current_runner_path() -> Path:
    arg0 = Path(sys.argv[0]).resolve()
    if arg0.exists():

```

```

        return arg0
    return THIS_FILE

def sha256_file(path: Path) -> str:
    if not path.exists() or not path.is_file():
        return ""
    digest = hashlib.sha256()
    with path.open("rb") as handle:
        for chunk in iter(lambda: handle.read(1024 * 1024), b''):
            digest.update(chunk)
    return digest.hexdigest()

def execution_provenance(outputs: list[dict[str, Any]], input_files: list[str] | None =
None) -> dict[str, Any]:
    runner = current_runner_path()
    artifacts = []
    for item in outputs:
        if isinstance(item, dict) and item.get("path"):
            artifacts.append(contract_path(item["path"]))
    return {
        "source_code_path": rel(runner),
        "source_code_sha256": sha256_file(runner),
        "helper_path": rel(THIS_FILE),
        "run_command": f"{Path(sys.executable).name} {rel(runner)}",
        "run_exit_code": 0,
        "input_files": [contract_path(path) for path in (input_files or [])],
        "output_artifacts": artifacts,
        "generated_at": now(),
    }

def normalize_task_type(task_type: str) -> str:
    text = str(task_type or "").lower()
    if any(key in text for key in ("预测", "回归", "forecast", "regression", "时间序列")):
        return "forecasting"
    if any(key in text for key in ("优化", "规划", "调度", "选址", "路径", "决策")):
        return "optimization"
    if any(key in text for key in ("评价", "排序", "权重", "综合", "topsis", "ahp", "熵
权")):
        return "evaluation"
    if any(key in text for key in ("分类", "识别", "判别", "classification")):
        return "classification"
    if any(key in text for key in ("聚类", "分群", "clustering")):
        return "clustering"
    if any(key in text for key in ("仿真", "机理", "动力学", "微分", "simulation")):
        return "simulation"
    return "general"

def load_json(path: Path, default: Any) -> Any:

```

```

    if not path.exists():
        return default
    try:
        return json.loads(path.read_text(encoding="utf-8"))
    except Exception:
        return default

def write_json(path: Path, data: Any) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    path.write_text(json.dumps(data, ensure_ascii=False, indent=2), encoding="utf-8")

def safe_slug(text: object) -> str:
    cleaned = "".join(ch.lower() if ch.isalnum() else "_" for ch in str(text))
    while "__" in cleaned:
        cleaned = cleaned.replace("__", "_")
    return cleaned.strip("_") or "item"

def find_cleaned_csv_files() -> list[Path]:
    if not DATA_CLEANED_DIR.exists():
        return []
    return sorted(DATA_CLEANED_DIR.rglob("*.csv"), key=lambda item:
item.as_posix().lower())

def read_dataframe(path: Path) -> pd.DataFrame:
    for encoding in ("utf-8-sig", "utf-8", "gbk", "gb18030"):
        try:
            return pd.read_csv(path, encoding=encoding)
        except Exception:
            continue
    raise RuntimeError(f"Unable to read CSV: {path}")

def load_first_dataset() -> tuple[pd.DataFrame | None, Path | None]:
    for path in find_cleaned_csv_files():
        try:
            df = read_dataframe(path)
        except Exception:
            continue
        if not df.empty:
            return df, path
    return None, None

def numeric_frame(df: pd.DataFrame) -> pd.DataFrame:
    converted = pd.DataFrame()
    for col in df.columns:
        values = pd.to_numeric(df[col], errors="coerce")
        if values.notna().sum() >= max(2, int(len(df) * 0.5)):

```

```

        converted[str(col)] = values
    return converted.replace([np.inf, -np.inf], np.nan)

def minmax(series: pd.Series) -> pd.Series:
    values = pd.to_numeric(series, errors="coerce").astype(float)
    lo = values.min()
    hi = values.max()
    if pd.isna(lo) or pd.isna(hi) or abs(hi - lo) < 1e-12:
        return pd.Series(np.zeros(len(values)), index=values.index)
    return (values - lo) / (hi - lo)

def round_float(value: Any, ndigits: int = 6) -> float | None:
    try:
        value = float(value)
    except Exception:
        return None
    if not math.isfinite(value):
        return None
    return round(value, ndigits)

def write_csv_table(path: Path, rows: list[dict[str, Any]]) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    if not rows:
        path.write_text("", encoding="utf-8")
        return
    fields: list[str] = []
    for row in rows:
        for key in row:
            if key not in fields:
                fields.append(key)
    with path.open("w", encoding="utf-8-sig", newline="") as handle:
        writer = csv.DictWriter(handle, fieldnames=fields)
        writer.writeheader()
        writer.writerows(rows)

def table_entry(question_id: str, table_id: str, title: str, purpose: str, path: Path,
status: str) -> dict[str, Any]:
    return {
        "table_id": table_id,
        "question_id": question_id,
        "title": title,
        "purpose": purpose,
        "path": rel(path),
        "source": "paper_output/code/modeling",
        "status": status,
    }

```

```

def upsert_question_contracts(
    question: dict[str, Any],
    result_summary: str,
    metrics: list[dict[str, Any]],
    tables: list[dict[str, Any]],
    conclusions: list[dict[str, Any]],
    outputs: list[dict[str, Any]],
    parameters: list[dict[str, Any]] | None = None,
    status: str = "scaffold_result_needs_review",
) -> None:
    RESULTS_DIR.mkdir(parents=True, exist_ok=True)
    TABLES_DIR.mkdir(parents=True, exist_ok=True)
    qid = str(question.get("question_id") or question.get("id") or "Q1")
    title = str(question.get("title") or qid)
    task_type = str(question.get("task_type") or "")
    main_model = str(question.get("main_model") or question.get("baseline_model") or "")

    model_results_path = RESULTS_DIR / "model_results.json"
    metrics_path = RESULTS_DIR / "metrics.json"
    conclusions_path = RESULTS_DIR / "conclusions.json"
    table_index_path = TABLES_DIR / "table_index.json"

    common = {
        "schema_version": "1.0",
        "generated_by": "paper_output/code/modeling/result_contract_io.py",
        "generated_at": now(),
    }

    model_results = load_json(model_results_path, {**common, "questions": []})
    metric_contract = load_json(metrics_path, {**common, "items": []})
    conclusion_contract = load_json(conclusions_path, {**common, "items": []})
    table_contract = load_json(table_index_path, {**common, "tables": []})

    for contract in (model_results, metric_contract, conclusion_contract, table_contract):
        contract.setdefault("schema_version", "1.0")
        contract["generated_at"] = now()

    normalized_outputs = normalize_outputs(outputs)
    result_item = {
        "question_id": qid,
        "title": title,
        "task_type": task_type,
        "result_type": normalize_task_type(task_type),
        "main_model": main_model,
        "baseline_model": question.get("baseline_model", ""),
        "result_summary": result_summary,
        "outputs": normalized_outputs,
        "parameters": parameters or [],
        "evidence_status": status,
        "status": status,
        "execution_provenance": execution_provenance(normalized_outputs),
    }

    questions = [item for item in model_results.get("questions", []) if

```

```

str(item.get("question_id")) != qid]
    questions.append(result_item)
    model_results["questions"] = questions

    metric_items = [item for item in metric_contract.get("items", []) if
str(item.get("question_id")) != qid]
    for metric in metrics:
        metric_items.append({"question_id": qid, "status": status, **metric})
    metric_contract["items"] = metric_items

    conclusion_items = [item for item in conclusion_contract.get("items", []) if
str(item.get("question_id")) != qid]
    conclusion_items.extend(conclusions)
    conclusion_contract["items"] = conclusion_items

    new_table_ids = {str(item.get("table_id")) for item in tables}
    table_items = [item for item in table_contract.get("tables", []) if
str(item.get("table_id")) not in new_table_ids]
    table_items.extend(tables)
    table_contract["tables"] = table_items
    notes = table_contract.setdefault("notes", [])
    note = "status=scaffold_result_needs_review 的表格来自自动脚手架，正式提交前应由 Agent 结
合真复核或改写。"
    if note not in notes:
        notes.append(note)

    write_json(model_results_path, model_results)
    write_json(metrics_path, metric_contract)
    write_json(conclusions_path, conclusion_contract)
    write_json(table_index_path, table_contract)

def no_data_result(question: dict[str, Any]) -> int:
    qid = str(question.get("question_id") or question.get("id") or "Q1")
    upsert_question_contracts(
        question=question,
        result_summary=f"{qid} 未检测到可用清洗数据。脚手架已保留，需 Agent 补充数据读取和专用建
模代码。",
        metrics=[{"metric_name": "data_available", "metric_role": "是否检测到清洗数据",
"value": 0, "unit": ""}],
        tables=[],
        conclusions=[{"question_id": qid, "conclusion_text": f"{qid} 需要补入真实数据和专用建
模代码后才能形成结论。", "evidence_status": "needs_real_modeling"}],
        outputs=[],
        status="needs_real_modeling",
    )
    return 0

def run_forecasting(question: dict[str, Any], df: pd.DataFrame, source_path: Path) -> int:
    qid = str(question.get("question_id") or "Q1")
    num = numeric_frame(df).dropna(axis=1, how="all")

```

```

if num.empty:
    return no_data_result(question)
target = num.columns[-1]
feature_cols = [col for col in num.columns if col != target]
y = num[target].fillna(num[target].median()).to_numpy(dtype=float)
if feature_cols:
    x = num[feature_cols].fillna(num[feature_cols].median()).to_numpy(dtype=float)
else:
    feature_cols = ["row_index"]
    x = np.arange(len(num), dtype=float).reshape(-1, 1)
design = np.column_stack([np.ones(len(x)), x])
split = max(1, int(len(design) * 0.75)) if len(design) >= 4 else len(design)
train_x = design[:split]
train_y = y[:split]
test_x = design[split:] if split < len(design) else design
test_y = y[split:] if split < len(design) else y
try:
    coef = np.linalg.lstsq(train_x, train_y, rcond=None)[0]
    pred_all = design @ coef
    pred_test = test_x @ coef
except Exception:
    coef = np.array([float(np.nanmean(train_y))])
    pred_all = np.full(len(y), coef[0])
    pred_test = np.full(len(test_y), coef[0])
residual = test_y - pred_test
rmse = float(np.sqrt(np.nanmean(residual ** 2))) if len(residual) else 0.0
mae = float(np.nanmean(np.abs(residual))) if len(residual) else 0.0
denom = np.where(np.abs(test_y) < 1e-12, np.nan, np.abs(test_y))
mape = float(np.nanmean(np.abs(residual) / denom) * 100) if np.isfinite(denom).any()
else 0.0
rows = [
    {"row_index": idx, "actual": round_float(actual), "predicted": round_float(pred),
    "residual": round_float(actual - pred)}
    for idx, (actual, pred) in enumerate(zip(y, pred_all))
]
table_id = f"table_{qid.lower()}_forecasting_scaffold"
table_path = TABLES_DIR / f"{table_id}.csv"
write_csv_table(table_path, rows)
upsert_question_contracts(
    question,
    result_summary=f"{qid} 已基于 `{rel(source_path)}` 生成预测/回归脚手架结果。当前自动选
择 `{target}` 作为目标列，特征列为 {'', ' '.join(map(str, feature_cols))}; Agent 需按题意复核目标
列、特征工程和评价口径。",
    metrics=[
        {"metric_name": "RMSE", "metric_role": "预测误差", "value": round_float(rmse),
        "unit": ""},
        {"metric_name": "MAE", "metric_role": "平均绝对误差", "value": round_float(mae),
        "unit": ""},
        {"metric_name": "MAPE", "metric_role": "相对误差", "value": round_float(mape),
        "unit": "%"},
    ],
    tables=[table_entry(qid, table_id, f"{qid} 预测结果脚手架表", "保存、和残差，供 Agent

```

```

二次替换为正式建模结果。", table_path, "scaffold_result_needs_review")],
    conclusions=[{"question_id": qid, "conclusion_text": f"{qid} 已形成可运行预测脚手架,
但目标列和特征列仍需按题意复核。", "evidence_status": "scaffold_result_needs_review"}],
    outputs=[{"name": "forecast_table", "path": rel(table_path)}],
    parameters=[{"name": "target_column", "value": str(target)}, {"name":
"feature_columns", "value": list(map(str, feature_cols))}],
    )
    return 0

def run_optimization(question: dict[str, Any], df: pd.DataFrame, source_path: Path) ->
int:
    qid = str(question.get("question_id") or "Q1")
    num = numeric_frame(df)
    if num.empty:
        return no_data_result(question)
    benefit_col = num.columns[0]
    cost_col = num.columns[1] if len(num.columns) > 1 else None
    score = minmax(num[benefit_col]).fillna(0)
    if cost_col is not None:
        score = score - minmax(num[cost_col]).fillna(0)
    ranked = pd.DataFrame({"row_index": num.index, "score": score})
    ranked = ranked.sort_values("score", ascending=False).reset_index(drop=True)
    rows = [{"rank": idx + 1, "row_index": int(row["row_index"]), "score":
round_float(row["score"])} for idx, row in ranked.iterrows()]
    table_id = f"table_{qid.lower()}_optimization_scaffold"
    table_path = TABLES_DIR / f"{table_id}.csv"
    write_csv_table(table_path, rows)
    upsert_question_contracts(
        question,
        result_summary=f"{qid} 已生成优化/规划脚手架结果。当前以 `{benefit_col}` 为收益代理指
标 · `{'`' + str(cost_col) + '`' 为成本代理指标') if cost_col is not None else '重置成本代理
指标'; Agent 需替换为正式目标函数、约束条件和求解器。",
        metrics=[
            {"metric_name": "objective_value", "metric_role": "目标函数代理值", "value":
round_float(ranked.iloc[0]["score"]) if not ranked.empty else None, "unit": ""},
            {"metric_name": "constraint_satisfaction_rate", "metric_role": "约束满足率",
"value": None, "unit": "%", "status": "to_be_filled"},
            {"metric_name": "selected_count", "metric_role": "候选方案数量", "value":
len(ranked), "unit": "rows"},
        ],
        tables=[table_entry(qid, table_id, f"{qid} 优化方案脚手架表", "保存代理得分排序, 供
Agent 替换为正式优化结果和约束校验。", table_path, "scaffold_result_needs_review")],
        conclusions=[{"question_id": qid, "conclusion_text": f"{qid} 已形成优化脚手架, 正式结
论需补充目标函数、约束和最优方案解释。", "evidence_status": "scaffold_result_needs_review"}],
        outputs=[{"name": "optimization_table", "path": rel(table_path)}],
        parameters=[{"name": "benefit_proxy", "value": str(benefit_col)}, {"name":
"cost_proxy", "value": str(cost_col or "")}],
    )
    return 0

```

```

def run_evaluation(question: dict[str, Any], df: pd.DataFrame, source_path: Path) -> int:
    qid = str(question.get("question_id") or "Q1")
    num = numeric_frame(df)
    if num.empty:
        return no_data_result(question)
    normalized = pd.DataFrame({col: minmax(num[col]).fillna(0) for col in num.columns})
    weights = np.ones(len(normalized.columns)) / max(1, len(normalized.columns))
    scores = normalized.to_numpy(dtype=float) @ weights
    ranked = pd.DataFrame({"row_index": num.index, "score": scores}).sort_values("score",
ascending=False).reset_index(drop=True)
    rows = [{"rank": idx + 1, "row_index": int(row["row_index"]), "score":
round_float(row["score"])} for idx, row in ranked.iterrows()]
    table_id = f"table_{qid.lower()}_evaluation_scaffold"
    table_path = TABLES_DIR / f"{table_id}.csv"
    write_csv_table(table_path, rows)
    upsert_question_contracts(
        question,
        result_summary=f"{qid} 已生成综合评价脚手架结果。当前对数值指标做 0-1 归一化并采用等权综
合得分；Agent 需按题意替换指标方向、权重方法和敏感性检验。",
        metrics=[
            {"metric_name": "top_score", "metric_role": "最高综合得分", "value":
round_float(ranked.iloc[0]["score"]) if not ranked.empty else None, "unit": ""},
            {"metric_name": "indicator_count", "metric_role": "参与评价指标数", "value":
len(normalized.columns), "unit": ""},
            {"metric_name": "weight_sensitivity", "metric_role": "权重敏感性", "value":
None, "unit": "", "status": "to_be_filled"}],
    ],
    tables=[table_entry(qid, table_id, f"{qid} 综合评价排序脚手架表", "保存代理综合得分和
排序，供 Agent 替换为正式评价模型结果。", table_path, "scaffold_result_needs_review")],
    conclusions=[{"question_id": qid, "conclusion_text": f"{qid} 已形成综合评价脚手架，正
式结论需补充指标体系、权重和稳定性分析。", "evidence_status":
"scaffold_result_needs_review"}],
    outputs=[{"name": "evaluation_table", "path": rel(table_path)}],
    parameters=[{"name": "indicator_columns", "value": list(map(str,
normalized.columns))}, {"name": "weight_method", "value": "equal_weight_scaffold"}],
    )
    return 0

def run_classification(question: dict[str, Any], df: pd.DataFrame, source_path: Path) ->
int:
    qid = str(question.get("question_id") or "Q1")
    num = numeric_frame(df)
    if num.empty:
        return no_data_result(question)
    score = minmax(num.iloc[:, 0]).fillna(0)
    threshold = float(score.median())
    predicted = np.where(score >= threshold, "class_high", "class_low")
    rows = [{"row_index": int(index), "predicted_class": str(label), "score":
round_float(value)} for index, label, value in zip(num.index, predicted, score)]
    table_id = f"table_{qid.lower()}_classification_scaffold"
    table_path = TABLES_DIR / f"{table_id}.csv"

```

```

write_csv_table(table_path, rows)
upsert_question_contracts(
    question,
    result_summary=f"{qid} 已生成分类/识别脚手架结果。当前使用 `{num.columns[0]}` 的归一化
    阈值形成二分类代理；Agent 需替换为真实标签、特征工程和分类器。",
    metrics=[
        {"metric_name": "accuracy", "metric_role": "分类准确率", "value": None, "unit":
        "%", "status": "to_be_filled"},
        {"metric_name": "f1_score", "metric_role": "F1 值", "value": None, "unit": "",
        "status": "to_be_filled"},
        {"metric_name": "class_count", "metric_role": "类别数量", "value":
        len(set(predicted)), "unit": ""},
    ],
    tables=[table_entry(qid, table_id, f"{qid} 分类结果脚手架表", "保存代理分类标签，供
    Agent 替换为正式分类模型结果。", table_path, "scaffold_result_needs_review")],
    conclusions=[{"question_id": qid, "conclusion_text": f"{qid} 已形成分类脚手架，正式结
    论需补充真实标签、混淆矩阵和分类指标。", "evidence_status": "scaffold_result_needs_review"}],
    outputs=[{"name": "classification_table", "path": rel(table_path)}],
    parameters=[{"name": "proxy_feature", "value": str(num.columns[0])}, {"name":
    "threshold", "value": round_float(threshold)}],
)
return 0

```

```

def run_clustering(question: dict[str, Any], df: pd.DataFrame, source_path: Path) -> int:
    qid = str(question.get("question_id") or "Q1")
    num = numeric_frame(df).fillna(0)
    if num.empty:
        return no_data_result(question)
    x = np.column_stack([minmax(num[col]).fillna(0).to_numpy(dtype=float) for col in
    num.columns])
    n = len(x)
    k = 1 if n < 2 else min(3, max(2, int(round(math.sqrt(n / 2)))))
    centroids = x[:k].copy()
    labels = np.zeros(n, dtype=int)
    for _ in range(30):
        distances = np.linalg.norm(x[:, None, :] - centroids[None, :, :], axis=2)
        new_labels = np.argmin(distances, axis=1)
        if np.array_equal(new_labels, labels):
            break
        labels = new_labels
        for cluster in range(k):
            if np.any(labels == cluster):
                centroids[cluster] = x[labels == cluster].mean(axis=0)
        compactness = float(np.mean([np.linalg.norm(x[i] - centroids[labels[i]]) for i in
        range(n)])) if n else 0.0
        rows = [{"row_index": int(index), "cluster": int(label) + 1} for index, label in
        enumerate(labels)]
        table_id = f"table_{qid.lower()}_clustering_scaffold"
        table_path = TABLES_DIR / f"{table_id}.csv"
        write_csv_table(table_path, rows)
        upsert_question_contracts(

```

```

        question,
        result_summary=f"{qid} 已生成聚类脚手架结果。当前对数值列归一化后进行 KMeans 代理聚类,
聚类数为 {k}; Agent 需结合题意解释群体特征并复核聚类数。",
        metrics=[
            {"metric_name": "silhouette_score", "metric_role": "轮廓系数", "value": None,
            "unit": "", "status": "to_be_filled"},
            {"metric_name": "cluster_count", "metric_role": "聚类数量", "value": k, "unit":
            ""},
            {"metric_name": "cluster_compactness", "metric_role": "簇内紧凑度代理", "value":
            round_float(compactness), "unit": ""},
        ],
        tables=[table_entry(qid, table_id, f"{qid} 聚类标签脚手架表", "保存自动聚类标签, 供
Agent 结合业务含义解释和复核。", table_path, "scaffold_result_needs_review")],
        conclusions=[{"question_id": qid, "conclusion_text": f"{qid} 已形成聚类脚手架, 正式结
论需补充聚类数选择依据和群体特征解释。", "evidence_status": "scaffold_result_needs_review"}],
        outputs=[{"name": "cluster_label_table", "path": rel(table_path)}],
        parameters=[{"name": "cluster_count", "value": k}, {"name": "feature_columns",
"value": list(map(str, num.columns))}],
    )
    return 0

def run_simulation(question: dict[str, Any], df: pd.DataFrame, source_path: Path) -> int:
    qid = str(question.get("question_id") or "Q1")
    num = numeric_frame(df)
    if num.empty:
        return no_data_result(question)
    target = num.columns[-1]
    y = num[target].fillna(num[target].median()).to_numpy(dtype=float)
    x = np.arange(len(y), dtype=float)
    coef = np.polyfit(x, y, 1) if len(y) >= 2 else np.array([0.0, float(y[0])])
    fitted = coef[0] * x + coef[1]
    rmse = float(np.sqrt(np.mean((y - fitted) ** 2))) if len(y) else 0.0
    future_x = np.arange(len(y), len(y) + 5, dtype=float)
    baseline = coef[0] * future_x + coef[1]
    rows = [
        {"step": step, "baseline": round_float(value), "low_scenario": round_float(value *
0.9), "high_scenario": round_float(value * 1.1)}
        for step, value in enumerate(baseline, start=1)
    ]
    table_id = f"table_{qid.lower()}_simulation_scaffold"
    table_path = TABLES_DIR / f"{table_id}.csv"
    write_csv_table(table_path, rows)
    upsert_question_contracts(
        question,
        result_summary=f"{qid} 已生成机理/仿真脚手架结果。当前对 `{target}` 建立线性趋势代理并
给出上下 10% 情景; Agent 需替换为正式机理方程或仿真模型。",
        metrics=[
            {"metric_name": "fit_error", "metric_role": "历史拟合误差", "value":
            round_float(rmse), "unit": ""},
            {"metric_name": "parameter_sensitivity", "metric_role": "参数敏感性", "value":
            10, "unit": "%"},

```

```

        {"metric_name": "scenario_change", "metric_role": "情景变化幅度", "value": 10,
"unit": "%"},
    ],
    tables=[table_entry(qid, table_id, f"{qid} 情景仿真脚手架表", "保存趋势代理与上下情景
结果, 供 Agent 替换为正式仿真输出。", table_path, "scaffold_result_needs_review")],
    conclusions=[{"question_id": qid, "conclusion_text": f"{qid} 已形成仿真脚手架, 正式结
论需补充机理参数和情景设定依据。", "evidence_status": "scaffold_result_needs_review"}],
    outputs=[{"name": "simulation_table", "path": rel(table_path)}],
    parameters=[{"name": "target_column", "value": str(target)}, {"name":
"linear_slope", "value": round_float(coef[0])}],
    )
    return 0

```

```

def run_general(question: dict[str, Any], df: pd.DataFrame, source_path: Path) -> int:
    qid = str(question.get("question_id") or "Q1")
    num = numeric_frame(df)
    if num.empty:
        return no_data_result(question)
    rows = [
        {"field": str(col), "mean": round_float(num[col].mean()), "std":
round_float(num[col].std()), "min": round_float(num[col].min()), "max":
round_float(num[col].max())}
        for col in num.columns
    ]
    table_id = f"table_{qid.lower()}_general_scaffold"
    table_path = TABLES_DIR / f"{table_id}.csv"
    write_csv_table(table_path, rows)
    upsert_question_contracts(
        question,
        result_summary=f"{qid} 已生成通用统计建模脚手架结果。当前根据 `{rel(source_path)}` 输
出字段摘要: Agent 需结合模型路线补充正式模型。",
        metrics=[
            {"metric_name": "core_score", "metric_role": "核心评价指标代理", "value":
round_float(num.iloc[:, 0].mean()), "unit": ""},
            {"metric_name": "baseline_comparison", "metric_role": "基线对比结果", "value":
None, "unit": "", "status": "to_be_filled"},
            {"metric_name": "robustness_check", "metric_role": "稳健性检查", "value":
len(num), "unit": "rows"},
        ],
        tables=[table_entry(qid, table_id, f"{qid} 通用统计脚手架表", "保存数值字段统计摘要,
供 Agent 作为正式模型输入参考。", table_path, "scaffold_result_needs_review")],
        conclusions=[{"question_id": qid, "conclusion_text": f"{qid} 已形成通用统计脚手架, 正
式结论需结合模型路线进一步补齐。", "evidence_status": "scaffold_result_needs_review"}],
        outputs=[{"name": "general_profile_table", "path": rel(table_path)}],
        parameters=[{"name": "numeric_columns", "value": list(map(str, num.columns))}],
    )
    return 0

```

```

def run_question_scaffold(question: dict[str, Any]) -> int:
    df, source_path = load_first_dataset()

```

```

if df is None or source_path is None:
    return no_data_result(question)
kind = normalize_task_type(str(question.get("task_type") or ""))
handlers = {
    "forecasting": run_forecasting,
    "optimization": run_optimization,
    "evaluation": run_evaluation,
    "classification": run_classification,
    "clustering": run_clustering,
    "simulation": run_simulation,
    "general": run_general,
}
handler = handlers.get(kind, run_general)
print(f"[modeling scaffold] {question.get('question_id', 'Q?')}")
task_type={question.get('task_type', '')} kind={kind} data={rel(source_path)}")
return handler(question, df, source_path)

```

D.7 paper_output/code/modeling/run_modeling.py

```

# Generated by MathModel Skill scaffold generator.
from __future__ import annotations

import hashlib
import json
import os
import platform
import subprocess
import sys
from datetime import datetime
from pathlib import Path

THIS_DIR = Path(__file__).resolve().parent

def detect_project_root() -> Path:
    for parent in THIS_DIR.parents:
        if parent.name == "paper_output":
            return parent.parent
    return Path.cwd().resolve()

PROJECT_ROOT = detect_project_root()
OUTPUT_DIR = PROJECT_ROOT / "paper_output"
RESULTS_DIR = OUTPUT_DIR / "results"
DATA_CLEANED_DIR = OUTPUT_DIR / "data_cleaned"
MODEL_RESULTS_FILE = RESULTS_DIR / "model_results.json"
RUN_MANIFEST_FILE = RESULTS_DIR / "run_manifest.json"

def configure_utf8_stdio() -> None:
    for stream in (sys.stdout, sys.stderr):

```

```

        try:
            stream.reconfigure(encoding="utf-8", errors="replace")
        except Exception:
            pass

def safe_print(text: object = "", *, end: str = "\n") -> None:
    message = str(text)
    try:
        print(message, end=end)
    except UnicodeEncodeError:
        encoding = getattr(sys.stdout, "encoding", None) or "utf-8"
        safe = message.encode(encoding, errors="replace").decode(encoding,
errors="replace")
        print(safe, end=end)

def now() -> str:
    return datetime.now().isoformat(timespec="seconds")

def rel(path: Path) -> str:
    try:
        return path.resolve().relative_to(PROJECT_ROOT).as_posix()
    except Exception:
        return str(path).replace("\\", "/")

def resolve(path_text: object) -> Path:
    path = Path(str(path_text or "").strip())
    if path.is_absolute():
        return path
    return PROJECT_ROOT / path

def load_json(path: Path, default):
    if not path.exists():
        return default
    try:
        return json.loads(path.read_text(encoding="utf-8"))
    except Exception:
        return default

def sha256(path: Path) -> str:
    if not path.exists() or not path.is_file():
        return ""
    digest = hashlib.sha256()
    with path.open("rb") as handle:
        for chunk in iter(lambda: handle.read(1024 * 1024), b''):
            digest.update(chunk)
    return digest.hexdigest()

```

```

def input_file_entries() -> list[dict[str, str]]:
    if not DATA_CLEANED_DIR.exists():
        return []
    entries = []
    for path in sorted(item for item in DATA_CLEANED_DIR.rglob("*") if item.is_file()):
        entries.append({"path": rel(path), "bytes": path.stat().st_size, "sha256":
sha256(path)})
    return entries

def normalize_path(path_text: object) -> str:
    if not path_text:
        return ""
    return rel(resolve(path_text))

def provenance_for_script(script: Path) -> tuple[list[str], list[str]]:
    data = load_json(MODEL_RESULTS_FILE, {})
    questions = data.get("questions") if isinstance(data, dict) else []
    qids: list[str] = []
    artifacts: list[str] = []
    script_rel = rel(script)
    if not isinstance(questions, list):
        return qids, artifacts
    for item in questions:
        if not isinstance(item, dict):
            continue
        provenance = item.get("execution_provenance")
        if not isinstance(provenance, dict):
            continue
        if normalize_path(provenance.get("source_code_path")) != script_rel:
            continue
        qid = str(item.get("question_id") or "").strip()
        if qid:
            qids.append(qid)
        for artifact in provenance.get("output_artifacts", []) or []:
            path_text = str(artifact)
            if path_text and path_text not in artifacts:
                artifacts.append(path_text)
    return qids, artifacts

def artifact_entries(paths: list[str]) -> list[dict[str, object]]:
    entries = []
    for path_text in paths:
        path = resolve(path_text)
        entries.append(
            {
                "path": normalize_path(path_text),
                "exists": path.exists(),

```

```

        "bytes": path.stat().st_size if path.exists() and path.is_file() else 0,
        "sha256": sha256(path),
    }
)
return entries

def write_run_manifest(runs: list[dict[str, object]]) -> None:
    RESULTS_DIR.mkdir(parents=True, exist_ok=True)
    manifest = {
        "schema_version": "1.0",
        "generated_by": "paper_output/code/modeling/run_modeling.py",
        "generated_at": now(),
        "status": "PASS" if all(item.get("returncode") == 0 for item in runs) else "FAIL",
        "runs": runs,
    }
    RUN_MANIFEST_FILE.write_text(json.dumps(manifest, ensure_ascii=False, indent=2),
    encoding="utf-8")

def main() -> int:
    configure_utf8_stdio()
    scripts = sorted(path for path in THIS_DIR.glob("q*_model.py") if
path.name[:1].lower() == "q")
    if not scripts:
        safe_print("No q*_model.py scripts found.")
        write_run_manifest([])
        return 0
    failures = 0
    runs: list[dict[str, object]] = []
    for script in scripts:
        safe_print(f"=== Running {script.name} ===")
        started_at = now()
        command = [sys.executable, str(script)]
        env = dict(os.environ)
        env["PYTHONIOENCODING"] = "utf-8"
        result = subprocess.run(
            command,
            cwd=str(THIS_DIR),
            env=env,
            text=True,
            encoding="utf-8",
            errors="replace",
            stdout=subprocess.PIPE,
            stderr=subprocess.STDOUT,
            check=False,
        )
        if result.stdout:
            safe_print(result.stdout, end="" if result.stdout.endswith("\n") else "\n")
        if result.returncode != 0:
            failures += 1
            safe_print(f"[warning] {script.name} exited with code {result.returncode}")

```

```

qids, artifacts = provenance_for_script(script)
runs.append(
    {
        "run_id": f"{script.stem}_{started_at.replace(':', '').replace('-',
''.replace('T', '_'))}",
        "script": rel(script),
        "script_sha256": sha256(script),
        "question_ids": qids,
        "command": f"{Path(sys.executable).name} {script.name}",
        "returncode": result.returncode,
        "status": "PASS" if result.returncode == 0 else "FAIL",
        "started_at": started_at,
        "finished_at": now(),
        "working_directory": rel(THIS_DIR),
        "environment": {
            "python": sys.version.split()[0],
            "implementation": platform.python_implementation(),
            "platform": platform.platform(),
        },
        "input_files": input_file_entries(),
        "output_artifacts": artifact_entries(artifacts),
        "stdout_tail": (result.stdout or "")[-4000:],
    }
)
write_run_manifest(runs)
safe_print(f"Run manifest written: {RUN_MANIFEST_FILE}")
if failures:
    safe_print(f"Completed with {failures} failed modeling script(s).")
    return 1
safe_print("All modeling scaffold scripts completed.")
return 0

if __name__ == "__main__":
    raise SystemExit(main())

```

D. 8 paper_output/code/visualization/generate_model_evidence.py

```

"""Generate reproducible AQ1-AQ4 figures."""
from __future__ import annotations

import csv
import hashlib
import json
import math
from datetime import datetime, timezone
from pathlib import Path

import matplotlib.pyplot as plt

ROOT = Path(__file__).resolve().parents[3]

```

```

OUT = ROOT / "paper_output"
TABLES = OUT / "tables"
FIGURES = OUT / "figures"

def read_csv(name: str) -> tuple[list[str], list[list[float]]]:
    with (TABLES / name).open(encoding="utf-8-sig", newline="") as handle:
        rows = list(csv.reader(handle))
        header = rows[0]
        values = []
        for row in rows[1:]:
            parsed = []
            for value in row:
                try:
                    parsed.append(float(value))
                except ValueError:
                    parsed.append(math.nan)
            if parsed and math.isfinite(parsed[0]):
                values.append(parsed)
        return header, values

def sha256(path: Path) -> str:
    return hashlib.sha256(path.read_bytes()).hexdigest()

def save(fig: plt.Figure, name: str) -> Path:
    path = FIGURES / name
    fig.tight_layout()
    fig.savefig(path, dpi=160, bbox_inches="tight")
    plt.close(fig)
    return path

def main() -> None:
    FIGURES.mkdir(parents=True, exist_ok=True)
    _, q1t = read_csv("table_aq1_temperature.csv")
    _, q1m = read_csv("table_aq1_moisture.csv")
    q2t_h, q2t = read_csv("table_aq2_temperature.csv")
    _, q2m = read_csv("table_aq2_moisture.csv")
    _, q3 = read_csv("table_aq3_drying_moisture.csv")
    _, q4 = read_csv("table_aq4_shrinkage_moisture.csv")

    # AQ1: temporal evolution of centre/surface temperature and moisture.
    fig, axes = plt.subplots(1, 2, figsize=(10, 4))
    t = [row[0] / 60 for row in q1t]
    axes[0].plot(t, [row[1] for row in q1t], label="center")
    axes[0].plot(t, [row[-1] for row in q1t], label="surface")
    axes[0].set(xlabel="Time (min)", ylabel="Temperature (°C)", title="AQ1 temperature field")
    axes[1].plot([row[0] / 60 for row in q1m], [row[1] for row in q1m], label="center")
    axes[1].plot([row[0] / 60 for row in q1m], [row[-1] for row in q1m], label="surface")

```

```

    axes[1].set(xlabel="Time (min)", ylabel="Dry-basis moisture", title="AQ1 moisture
field")
    for axis in axes:
        axis.legend(); axis.grid(alpha=0.25)
    p1 = save(fig, "fig_aq1_fields.png")

# AQ2: radial profiles at each reported time.
fig, axes = plt.subplots(1, 2, figsize=(10, 4))
radius = [float(label.split())[0]) for label in q2t_h[1:]]
for row in q2t:
    axes[0].plot(radius, row[1:], marker="o", label=f"{row[0]:g} h")
for row in q2m:
    axes[1].plot(radius, row[1:], marker="o", label=f"{row[0]:g} h")
axes[0].set(xlabel="Radius (cm)", ylabel="Temperature (°C)", title="AQ2 radial
temperature")
axes[1].set(xlabel="Radius (cm)", ylabel="Dry-basis moisture", title="AQ2 radial
moisture")
for axis in axes:
    axis.legend(fontsize=7); axis.grid(alpha=0.25)
p2 = save(fig, "fig_aq2_profiles.png")

# AQ3: whole-domain maximum versus threshold.
fig, axis = plt.subplots(figsize=(6, 4))
hours = [row[0] for row in q3]
maxima = [max(value for value in row[1:] if math.isfinite(value)) for row in q3]
axis.plot(hours, maxima, marker="o", label="domain maximum")
axis.axhline(0.15, color="crimson", linestyle="--", label="threshold 0.15")
axis.set(xlabel="Time (h)", ylabel="Dry-basis moisture", title="AQ3 drying threshold")
axis.grid(alpha=0.25); axis.legend()
p3 = save(fig, "fig_aq3_threshold.png")

# AQ4: moving boundary and moisture profiles.
fig, axes = plt.subplots(1, 2, figsize=(10, 4))
hours = [row[0] for row in q4]
axes[0].plot(hours, [row[-1] for row in q4], marker="o", color="black")
axes[0].set(xlabel="Time (h)", ylabel="Radius (cm)", title="AQ4 shrinking domain")
for row in q4:
    values = [value for value in row[1:5] if math.isfinite(value)]
    axes[1].plot(range(len(values)), values, marker="o", label=f"{row[0]:g} h")
axes[1].set(xlabel="Radial node", ylabel="Dry-basis moisture", title="AQ4 moisture
profiles")
axes[1].legend(fontsize=7); axes[0].grid(alpha=0.25); axes[1].grid(alpha=0.25)
p4 = save(fig, "fig_aq4_shrinkage.png")

# Numerical validation is produced by run_numerical_validation.py; keep it separate
from plotting.
source_map = {
    "fig_aq1_fields": (p1, ["AQ1"], "table_aq1_temperature.csv"),
    "fig_aq2_profiles": (p2, ["AQ2"], "table_aq2_temperature.csv"),
    "fig_aq3_threshold": (p3, ["AQ3"], "table_aq3_drying_moisture.csv"),
    "fig_aq4_shrinkage": (p4, ["AQ4"], "table_aq4_shrinkage_moisture.csv"),
}

```

```

    existing = json.loads((OUT / "figure_index.json").read_text(encoding="utf-8")) if (OUT
/ "figure_index.json").exists() else {}
    index = {"schema_version": "1.0", "generated_at":
datetime.now(timezone.utc).isoformat(), "generated_by":
"paper_output/code/visualization/generate_model_evidence.py", "figures":
existing.get("figures", [])}
    for fid, source, qids in (("fig_a_environment", "a_environment.csv", ["AQ1", "AQ2",
"AQ3", "AQ4"]), ("fig_a_radius", "a_radius.csv", ["AQ4"])):
        path = OUT / "figures" / f"{fid}.png"
        data = OUT / "data_cleaned" / source
        if path.exists() and data.exists() and not any(x.get("figure_id") == fid for x in
index["figures"]):
            index["figures"].append({"figure_id": fid, "title": fid, "path":
path.relative_to(ROOT).as_posix(), "expected_path": path.relative_to(ROOT).as_posix(),
"question_ids": qids, "source_data": data.relative_to(ROOT).as_posix(), "source_sha256":
sha256(data), "sha256": sha256(path), "bytes": path.stat().st_size, "status": "generated",
"ok": True, "exists": True, "placeholder": False, "evidence_type": "input_observation"})
            for figure_id, (path, qids, source) in source_map.items():
                item = {"figure_id": figure_id, "title": figure_id, "path":
path.relative_to(ROOT).as_posix(), "expected_path": path.relative_to(ROOT).as_posix(),
"question_ids": qids, "question_id": qids[0], "source_data":
f"paper_output/tables/{source}", "source_sha256": sha256(TABLES / source), "sha256":
sha256(path), "bytes": path.stat().st_size, "status": "generated", "ok": True, "exists":
True, "placeholder": False, "evidence_type": "model_result"}
                index["figures"] = [x for x in index["figures"] if x.get("figure_id") !=
figure_id] + [item]
            (OUT / "figure_index.json").write_text(json.dumps(index, ensure_ascii=False,
indent=2), encoding="utf-8")

if __name__ == "__main__":
    main()

```

D.9 paper_output/code/visualization/run_numerical_validation.py

```

"""Run real solver comparisons for S6 convergence and sensitivity evidence."""
from __future__ import annotations

import hashlib
import json
import sys
import subprocess
from datetime import datetime, timezone
from pathlib import Path

import numpy as np
import pandas as pd

ROOT = Path(__file__).resolve().parents[3]
sys.path.insert(0, str(ROOT / "paper_output" / "code" / "modeling"))
from core_solver import simulate_fvm_fixed, simulate_fvm_moving

```

```

DATA = ROOT / "paper_output" / "data_cleaned"
OUT = ROOT / "paper_output" / "qa" / "model_validation_checks.json"

def digest(path: Path) -> str:
    return hashlib.sha256(path.read_bytes()).hexdigest()

def env():
    frame = pd.read_csv(DATA / "a_environment.csv")
    return (frame.time_s.to_numpy(float), frame.temperature_K.to_numpy(float),
    frame.air_moisture_kgkg.to_numpy(float))

def max_common_difference(a: np.ndarray, b: np.ndarray, n_a: int, n_b: int) -> float:
    ra = (np.arange(n_a) + 0.5) * 0.02 / n_a
    rb = (np.arange(n_b) + 0.5) * 0.02 / n_b
    common = np.linspace(ra[0], ra[-1], n_a)
    return float(np.max(np.abs(np.interp(common, rb, b) - np.interp(common, ra, a))))

def worker(kind: str, n: int, dt: float, total: int, output: Path) -> dict[str,
np.ndarray]:
    code = (ROOT / "paper_output" / "code" / "visualization" / "solver_worker.py")
    subprocess.run([sys.executable, str(code), kind, str(n), str(dt), str(total),
str(output)], check=True)
    with np.load(output) as data:
        return {key: data[key].copy() for key in data.files}

def main() -> None:
    tmp = ROOT / "paper_output" / "qa" / "_validation_tmp.npz"
    b0 = worker("fixed1", 80, 1.0, 1800, tmp); T0, C0 = b0["T"], b0["C"]
    bf = worker("fixed1", 160, 1.0, 1800, tmp); Tf, Cf = bf["T"], bf["C"]
    bt = worker("fixed1", 80, 0.5, 1800, tmp); Tt, Ct = bt["T"], bt["C"]
    base = {"N": 80, "dt_s": 1.0, "total_seconds": 1800, "formula_mode": 1}
    spatial = {"N": 160, "dt_s": 1.0, "total_seconds": 1800, "formula_mode": 1}
    temporal = {"N": 80, "dt_s": 0.5, "total_seconds": 1800, "formula_mode": 1}
    convergence = {
        "tolerance": 0.05,
        "baseline": {"config": base},
        "baseline_terminal_center_surface": {"T_C": [float(T0[0] - 273.15), float(T0[-1] -
273.15)], "C": [float(C0[0]), float(C0[-1])]},
        "spatial_refinement": {"config": spatial, "max_abs_T_K": max_common_difference(T0,
Tf, 80, 160), "max_abs_C": max_common_difference(C0, Cf, 80, 160)},
        "temporal_refinement": {"config": temporal, "max_abs_T_K": float(np.max(np.abs(T0
- Tt))), "max_abs_C": float(np.max(np.abs(C0 - Ct)))},
    }
    convergence["pass"] = all(value <= convergence["tolerance"] for key in
("spatial_refinement", "temporal_refinement") for value in (convergence[key]
["max_abs_T_K"], convergence[key]["max_abs_C"]))

```

```

te, Te, Ce = env()
tail_T = float(np.mean(Te[-10:])); tail_C = float(np.mean(Ce[-10:]))
boundary_total = 206433
bd = worker("fixed2", 80, 1.0, boundary_total, tmp); be = worker("tail2", 80, 1.0,
boundary_total, tmp)
end_d, end_e = float(bd["end"]), float(be["end"])
boundary = {"tolerance_relative": 0.02, "default_boundary": "hold_last_value",
"tail_extension": {"tail_samples": 10, "T_air_K": tail_T, "C_air": tail_C, "end_time_s":
boundary_total}, "default_threshold_time_s": float(end_d),
"tail_extension_threshold_time_s": float(end_e)}
boundary["relative_difference"] = abs(end_e - end_d) / end_d
boundary["pass"] = boundary["relative_difference"] <= boundary["tolerance_relative"]

total = 3600
fixed = worker("fixed3", 80, 1.0, total, tmp); moving = worker("moving3", 80, 1.0,
total, tmp)
Tfix, Cfix, Tmov, Cmov = fixed["T"], fixed["C"], moving["T"], moving["C"]
degeneration = {"config": {"N": 80, "dt_s": 1.0, "total_seconds": total,
"formula_mode": 3, "radius_m": 0.02}, "tolerance": 1e-10, "terminal_max_abs_T_K":
float(np.max(np.abs(Tfix - Tmov))), "terminal_max_abs_C": float(np.max(np.abs(Cfix -
Cmov)))}
degeneration["pass"] = degeneration["terminal_max_abs_T_K"] <=
degeneration["tolerance"] and degeneration["terminal_max_abs_C"] <=
degeneration["tolerance"]

result = {"schema_version": "1.0", "generated_by":
"paper_output/code/visualization/run_numerical_validation.py", "generated_at":
datetime.now(timezone.utc).isoformat(), "inputs": {"environment": {"path":
"paper_output/data_cleaned/a_environment.csv", "sha256": digest(DATA /
"a_environment.csv")}, "solver": {"path": "paper_output/code/modeling/core_solver.py",
"sha256": digest(ROOT / "paper_output/code/modeling/core_solver.py")},
"validation_script": {"path":
"paper_output/code/visualization/run_numerical_validation.py", "sha256":
digest(Path(__file__))}, "worker": {"path":
"paper_output/code/visualization/solver_worker.py", "sha256":
digest(Path(__file__).with_name("solver_worker.py"))}}, "checks": {"aq1_convergence":
convergence, "aq3_boundary_extension": boundary, "aq4_fixed_moving_degeneracy":
degeneration}}
result["status"] = "PASS" if all(item["pass"] for item in result["checks"].values())
else "FAIL"
OUT.write_text(json.dumps(result, ensure_ascii=False, indent=2), encoding="utf-8")
metrics_path = ROOT / "paper_output" / "results" / "metrics.json"
metrics = json.loads(metrics_path.read_text(encoding="utf-8"));
metrics["validation_generated_by"] =
"paper_output/code/visualization/run_numerical_validation.py"; metrics["generated_by"] =
"paper_output/code/modeling/result_contract_io.py"; metrics["items"] = [x for x in
metrics["items"] if not str(x.get("metric_name", "")).startswith("validation_")]
values = [("AQ1", "aq1_spatial_T_max_abs", convergence["spatial_refinement"]
["max_abs_T_K"], "K"), ("AQ1", "aq1_spatial_C_max_abs", convergence["spatial_refinement"]
["max_abs_C"], "kg/kg"), ("AQ1", "aq1_spatial_pass", int(convergence["pass"]), "boolean"),
("AQ1", "aq1_temporal_T_max_abs", convergence["temporal_refinement"]["max_abs_T_K"], "K"),
("AQ1", "aq1_temporal_C_max_abs", convergence["temporal_refinement"]["max_abs_C"],

```

```

"kg/kg"), ("AQ1", "aq1_temporal_pass", int(convergence["pass"]), "boolean"), ("AQ3",
"aq3_default_threshold_time_s", boundary["default_threshold_time_s"], "s"), ("AQ3",
"aq3_tail_mean_threshold_time_s", boundary["tail_extension_threshold_time_s"], "s"),
("AQ3", "aq3_boundary_relative_difference", boundary["relative_difference"], "fraction"),
("AQ3", "aq3_boundary_pass", int(boundary["pass"]), "boolean"), ("AQ4",
"aq4_fixed_moving_T_max_abs", degeneration["terminal_max_abs_T_K"], "K"), ("AQ4",
"aq4_fixed_moving_C_max_abs", degeneration["terminal_max_abs_C"], "kg/kg"), ("AQ4",
"aq4_fixed_moving_pass", int(degeneration["pass"]), "boolean")]
    for qid, name, value, unit in values:
        metrics["items"].append({"question_id": qid, "status": "computed", "metric_name":
f"validation_{name}", "metric_role": "validation", "value": value, "unit": unit, "source":
"paper_output/qa/model_validation_checks.json"})
        metrics_path.write_text(json.dumps(metrics, ensure_ascii=False, indent=2),
encoding="utf-8")
    if tmp.exists(): tmp.unlink()

if __name__ == "__main__":
    main()

```

D. 10 paper_output/code/visualization/solver_worker.py

```

from __future__ import annotations
import sys
from pathlib import Path
import numpy as np
import pandas as pd
sys.path.insert(0, str(Path(__file__).resolve().parents[1] / "modeling"))
from core_solver import simulate_fvm_fixed, simulate_fvm_moving

root = Path(__file__).resolve().parents[3]
d = pd.read_csv(root / "paper_output/data_cleaned/a_environment.csv")
t, T, C = d.time_s.to_numpy(float), d.temperature_K.to_numpy(float),
d.air_moisture_kgkg.to_numpy(float)
kind, n, dt, total, out = sys.argv[1], int(sys.argv[2]), float(sys.argv[3]),
int(sys.argv[4]), Path(sys.argv[5])
if kind == "tail2":
    t = np.r_[t, np.arange(t[-1] + 60, total + 1, 60)]; T = np.r_[T, np.full(len(t) -
len(d), T[-10:].mean())]; C = np.r_[C, np.full(len(t) - len(d), C[-10:].mean())]
if kind.startswith("moving"):
    x = simulate_fvm_moving(t, T, C, np.array([0., float(total)]), np.array([.02, .02]),
total, sample_every_s=3600, N=n, dt=dt, stop_at_cmax=-1.)
    np.savez(out, T=x[2][-1], C=x[3][-1], end=x[4])
else:
    mode = int(kind[-1]); sample = 1800 if dt < 1.0 else (60 if total > 10000 else 1); x =
simulate_fvm_fixed(t, T, C, total, mode, sample_every_s=sample, N=n, R=.02, h=25.,
hm=8e-7, dt=dt, stop_at_cmax=.15 if mode == 2 else -1.)
    np.savez(out, T=x[1][-1], C=x[2][-1], end=x[3])

```

D. 末 运行依赖

```
numpy>=2.0,<3  
numba>=0.60,<1  
pandas>=2.2,<4  
matplotlib>=3.8,<4  
seaborn>=0.13,<1  
requests>=2.32,<3  
python-docx>=1.1,<2  
pypdf>=5,<7  
openpyxl>=3.1,<4  
xlrd>=2,<3  
lxml>=5,<7  
latex2mathml>=3.77,<4
```